Lecture 01: Computer programming

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 1. Learning outcome: CLO-1.

Learning objectives

- Explain how a program solves a problem

- Describe structured, object-oriented and functional approaches

- Write and trace a finite algorithm

Concepts explained

Programs and algorithms

A program expresses instructions in a programming language. An algorithm gives precise steps for a task and must terminate for its intended inputs.

```
Task: calculate the cost of 3 notebooks at Rs 120 each.
Inputs: quantity, unit price
Process: multiply
Output: Rs 360
```

Input, processing and output

Inputs represent the information a problem supplies. Processing transforms that information. Outputs answer the original question.

```
READ quantity
READ unitPrice
total = quantity * unitPrice
DISPLAY total
```

Programming paradigms

Structured programming organises sequence, selection and repetition. Object-oriented programming groups state and behaviour in objects. Functional programming emphasises computing results from functions.

```
Structured: calculate a bill step by step.
Object-oriented: a Cart holds items and calculates its total.
Functional: calculate a new total from supplied prices.
```

Problem-solving workflow

State the required result and input constraints before coding. Work a small example manually, write an algorithm, then test the implementation.

```
Requirement: average two marks in 0..100.
Example: 60 and 80 gives 70.
Boundary: 0 and 100 gives 50.
```

Precision in an algorithm

A computer follows the stated rule even when the rule is unsuitable. An instruction such as "calculate the time" leaves the units and formula unspecified. A useful algorithm defines input, calculation and output precisely.

```
Ambiguous: convert the time.
Precise: multiply hours by 60, then add minutes.
```

Testing the model before coding

A manually solved case checks whether the formula represents the task. A zero case checks that the algorithm behaves sensibly at the smallest allowed input. Invalid inputs require a stated response before implementation.

```
2 h 35 min = 120 + 35 = 155 min
0 h 0 min = 0 min
2 h 75 min violates the stated minute range.
```

Key distinctions

Approach	Main organising idea	Example in a billing program
Structured	Ordered steps and control flow	Read items, calculate, print
Object-oriented	Objects with state and behaviour	A Cart calculates its total
Functional	Functions transform inputs	A function returns a total

First worked example

```
int quantity = 3;
int unitPrice = 120;
int total = quantity * unitPrice;
System.out.println(total);
```

Expected result and interpretation:

```
360
quantity=3 and unitPrice=120 remain unchanged.
total receives the product 360.
```

Guided problem and trace

Write pseudocode for converting hours and minutes into total minutes. Trace 2 hours 35 minutes and 0 hours 0 minutes.

1. Accept nonnegative hours and minutes in 0..59.

2. Convert only the hours to minutes using hours * 60.

3. Add the remaining minutes and label the output unit.

```
int hours = 2;
int minutes = 35;
int totalMinutes = hours * 60 + minutes;
System.out.println(totalMinutes + " minutes");
```

Step	Expression	Value
Read hours	hours	2
Read minutes	minutes	35
Convert hours	hours * 60	120
Combine	120 + 35	155
Report	totalMinutes	155 minutes

Expected output:

```
155 minutes
```

From a requirement to a result

A useful algorithm makes the meaning and units of every stage explicit.

Relevant labels: Requirement; Inputs; Calculation; Output

Case	Inputs	Expected area
Ordinary rectangle	width 8, height 3	24 square units
Zero width	width 0, height 3	0 square units
Outside the domain	width -2, height 3	Reject negative length

Additional worked example: Payroll as an algorithm

Identify the two inputs before writing Java: hours worked and hourly rate. The basic model multiplies hours by rate; overtime is a different requirement. Work a numeric case, generalise the steps, then test zero and ordinary inputs.

Calculate basic gross pay for 35 hours at 20 currency units per hour. State the input and output units.

Input: Fixed values in the example.

```
double hours = 35;
double rate = 20;
double grossPay = hours * rate;
System.out.printf(java.util.Locale.US, "Gross pay: %.2f%n", grossPay);
```

Hours	Rate	Gross pay
35	20	700.00
0	20	0.00
40	20	800.00

Expected output:

```
Gross pay: 700.00
```

Independent practice

1. Design pseudocode for rectangle area. Reject a negative side; allow a zero side. Trace (8,3), (0,3), and (-2,3).

2. What changes when the requirement introduces an overtime rate after 40 hours?

3. Debugging: A student calculates totalMinutes = hours + minutes * 60. Use 2 hours 35 minutes to explain the error.

4. Check your understanding: How does an algorithm differ from a program? Why is a single successful test insufficient?

5. Homework: Write an algorithm for a rectangle perimeter. Give normal, zero-size and invalid-input examples.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_01_Computer_programming.tex and PDF/Lecture_01_Computer_programming.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture01.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-2.pptx, slides 27, 29, 30, 31, 33. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 02: Source files and execution

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 1. Learning outcome: CLO-1.

Learning objectives

- Create and compile a Java source file

- Explain the Java execution pipeline

- Classify syntax, runtime and logic errors

Concepts explained

A first Java source file

A public class name matches its .java filename. The conventional entry point is public static void main(String[] args).

```
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello, Java");
  }
}
```

Compilation and execution

javac translates source code into JVM bytecode. java starts the JVM, which loads and links classes, then executes the program.

```
javac Hello.java
java Hello

Hello.java   source
Hello.class  bytecode
```

Three error categories

Syntax and type errors prevent successful compilation. Runtime exceptions interrupt execution. Logic errors produce an incorrect result even when execution finishes.

```
Syntax: missing ;
Runtime: Integer.parseInt("cat")
Logic: area = length + width
```

Files and working directories

A terminal command runs relative to its working directory. An editor or IDE helps save, build and run code, but the Java compiler still enforces language rules.

```
1. Create a course folder.
2. Save Hello.java as plain text.
3. Open a terminal in that folder.
4. Compile, then run.
```

What compilation checks

The compiler checks syntax and type consistency before producing bytecode. It cannot generally decide whether your formula answers the problem correctly. A successful build therefore needs a separate execution and result check.

```
int n = "five";  // incompatible types
int area = 5 + 3; // legal, but wrong for a 5 by 3 area
```

Reading an error message

A diagnostic normally identifies a file, line and kind of problem. Start with the earliest relevant error because one mistake can trigger several messages. Recompile after a small correction to see which messages remain.

```
Missing semicolon near a println statement:
Check that statement and the previous line.
```

Key distinctions

Stage	Input	Result
Edit	Plain-text source	Welcome.java
Compile	javac Welcome.java	Welcome.class or diagnostics
Load and link	java Welcome	JVM prepares classes
Execute	main statements	Console output

First worked example

```
System.out.println("Welcome to CSC103");
System.out.println(7 + 5);
```

Expected result and interpretation:

```
Welcome to CSC103
12
The first statement prints text. The second prints an arithmetic result.
```

Guided problem and trace

Create Welcome.java. Print your name and the result of 9 * 4 on separate lines. Compile and execute it.

1. Create a source file whose public class matches its filename.

2. Put the two output statements inside main.

3. Compile the file, then run the class without a .class extension.

```
System.out.println("Amina");
System.out.println(9 * 4);
```

Statement	Evaluation	Console line
println("Amina")	String literal	Amina

Statement	Evaluation	Console line
println(9 * 4)	9 multiplied by 4	36
End of main	Normal return	No additional line

Expected output:

```
Amina
36
```

The Java execution pipeline

Compilation and execution are different stages; each needs its own check.

Relevant labels: Source .java; javac compiler; Bytecode .class; JVM execution

Observation	Stage	Next check
Missing semicolon	Compilation	Inspect this and the preceding line
Class not found	Launch / loading	Class name and classpath
Wrong printed total	Execution / logic	Formula and test input

Additional worked example: Java program phases and runtime roles

javac checks source and emits class-file bytecode; the java launcher starts execution through a JVM. Loading, verification and execution are distinct runtime responsibilities. JIT compilation works on bytecode, not Java source. Use a JDK for development. Modern JDK layouts no longer use the old rt.jar structure shown in the source.

Build and run a small program, then identify which phase detects a missing semicolon and which phase prints the greeting.

Input: Fixed values in the example.

```
System.out.println("Hello, World");
```

Stage	Artifact or action	Evidence
Compile	javac IsbLecture02.java	Class file or diagnostic
Load and verify	Prepare class bytecode	Valid class structure required
Execute	Run main	Hello, World

Expected output:

```
Hello, World
```

Independent practice

1. A public class named Receipt is saved as Bill.java. After renaming it, the program prints 5 + 3 when the required area is 5 by 3. Identify both corrections and the commands.

2. Does bytecode verification prove that the formula in a program is correct?

3. Debugging: File name: Start.java
public class Begin { }

4. Check your understanding: Does java Hello compile Hello.java in the demonstrated two-step workflow? What file does javac create?

5. Homework: Create one example of each error category. Record the error message or wrong result and its correction.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_02_Source_files_and_execution.tex and PDF/Lecture_02_Source_files_and_execution.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture02.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-1.KI.pptx, slides 18, 25, 26, 27, 28, 29. Corrected the legacy rt.jar/JRE description and source-versus-bytecode wording. Build commands use the supplied IsbLecture02 filename.

Lecture 03: Java syntax and program style

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Liang Ch 2. Learning outcome: CLO-1.

Learning objectives

- Distinguish tokens, identifiers and reserved words

- Explain syntax versus meaning

- Apply consistent names, indentation and comments

Concepts explained

Tokens and special symbols

Tokens include keywords, identifiers, literals and operators. Braces delimit blocks. Parentheses group expressions and arguments. A semicolon ends many statements.

```
int score = 75;
System.out.println(score);
```

Identifiers and reserved words

Choose meaningful names such as totalMarks. Names are case-sensitive and cannot be reserved words. Class names conventionally use PascalCase and variables use camelCase.

```
Valid: studentCount, marks2
Invalid: 2marks, class
Different: score and Score
```

Comments and documentation

A line comment starts with //. A block comment uses /* ... */. Documentation comments use /** ... */ before declarations. Useful comments explain intent or constraints.

```
// Convert whole hours to minutes.
int totalMinutes = hours * 60;
```

Syntax, semantics and style

Syntax defines legal structure. Semantics concerns what the program means. Indentation helps people see nesting. Descriptive names help people understand intent.

```
int length = 5;
int width = 3;
int area = length * width;
```

A statement can be legal and still be wrong

The compiler accepts an expression that satisfies Java grammar and type rules. Correctness also requires the expression to match the intended quantity. Names and units help a reader check this connection.

```
For sides 6 cm and 4 cm:
Area = 6 * 4 = 24 cm^2
Perimeter = 2 * (6 + 4) = 20 cm
```

Documentation that explains decisions

A purpose comment states what the program calculates. A constraint comment explains the assumptions behind a calculation. Changing code and leaving an old comment creates misleading documentation.

```
Useful: Side lengths are in centimetres.
Weak: Add length and width.
The expression already shows the addition.
```

Key distinctions

Token	Category	Purpose
int	Keyword	Declares an integer type
perimeter	Identifier	Names the result
2	Literal	Supplies a fixed value
*	Operator	Multiplies operands
;	Separator	Ends the declaration statement

First worked example

```
// A rectangle with sides in centimetres.
int length = 5;
int width = 3;
int area = length * width;
System.out.println(area);
```

Expected result and interpretation:

```
15
All variable names describe their purpose.
The comment identifies the unit of measurement.
```

Guided problem and trace

Rewrite: int a=6;int b=4;System.out.println(2*(a+b)); Use names, spacing and a useful comment.

1. Rename a and b so that their roles are visible.

2. Group length + width before doubling it.

3. Use spacing, separate statements and a comment that identifies the measurement unit.

```
// Side lengths are in centimetres.
int length = 6;
int width = 4;
int perimeter = 2 * (length + width);
System.out.println(perimeter + " cm");
```

Expression	Meaning	Value
length	First side	6 cm

Expression	Meaning	Value
width	Second side	4 cm
length + width	Adjacent sides	10 cm
2 * (length + width)	All four sides	20 cm

Expected output:

```
20 cm
```

Read an expression as a syntax tree

In 2 * (length + width), the grouped sum supplies one operand of multiplication.

Relevant labels: *; +; 2; length; width

Expression	length=6, width=4	Interpretation
2 * (length + width)	20	Perimeter
2 * length + width	16	Only length is doubled
length * width	24	Area

Additional worked example: Repairing the Welcome program

The course uses public static void main(String[] args) as its entry point. Main and main are different identifiers. A string literal needs matching double quotes; a character literal cannot hold a sentence. Balance class and method braces, then use indentation to expose their nesting.

Repair the source activity containing public void Main and an incorrectly quoted greeting. Print the greeting once.

Input: Fixed values in the example.

```
System.out.println("Welcome to Java!");
```

Fault	Correction	Why
Main	main	Names are case-sensitive
Missing static	Add static to main	Conventional launch entry point
Mismatched quotes	Use double quotes	The greeting is a String

Expected output:

```
Welcome to Java!
```

Independent practice

1. Rewrite an unclear perimeter expression using named variables, parentheses and a units label. Explain why area and perimeter need different units.

2. Extend the corrected program to print the greeting five times without using loops yet.

3. Debugging: int class = 20;
System.out.println(Class);

4. Check your understanding: Is total marks a legal variable name? Does indentation change the meaning of braces?

5. Homework: Format your previous program. Add one purpose comment and explain why each variable name is meaningful.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_03_Java_syntax_and_program_style.tex and PDF/Lecture_03_Java_syntax_and_program_style.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture03.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-3.pptx, slides 20, 21, 24, 25. The intentionally faulty source activity is presented with a corrected, compilable full class.

Lecture 04: Variables and console input

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Liang Ch 2. Learning outcome: CLO-1.

Learning objectives

- Declare and initialise variables and constants

- Read numeric and character input

- Trace assignment as a change of stored value

Concepts explained

Variables and initialisation

A variable has a declared type and a name. Initialisation supplies its first value. A local variable must receive a value before a read.

```
int seats = 30;
seats = 28;
final int ROOM_LIMIT = 40;
```

Assignment copies a value

The right-hand expression is evaluated before assignment. For primitive variables, assigning one variable to another copies the current primitive value.

```
int a = 7;
int b = a;
a = 9;
// b is still 7
```

Scanner input

Create one Scanner for console input. nextInt reads an integer token and nextDouble reads a decimal token. The entered token must match the requested type.

```
java.util.Scanner in =
    new java.util.Scanner(System.in);
int age = in.nextInt();
```

Reading one character

Scanner has no nextChar method. next().charAt(0) reads the first UTF-16 code unit of the next token. For this exercise, input is one basic Latin character.

```
String token = in.next();
char choice = token.charAt(0);
```

Choosing a type from the data

A quantity that counts whole items fits an integer type. A price with a fractional part needs a suitable numeric representation. For introductory arithmetic, double is convenient, but exact financial applications need decimal handling.

```
quantity: 3
price: 12.5
category: A
Each input has a different role and type.
```

Console input is a sequence of tokens

Scanner reads the next available token when a next method succeeds. The program must request the token type in the order the user supplies it. A character read through next().charAt(0) uses only the first character of that token.

```
Input tokens: 3 12.5 A
nextInt consumes 3
nextDouble consumes 12.5
next consumes A
```

Key distinctions

Declaration	Initial value	May be reassigned?
int quantity = 3	3	Yes
double price = 12.5	12.5	Yes
char category = 'A'	A	Yes
final int LIMIT = 20	20	No after initial assignment

First worked example

```
int initial = 10;
int remaining = initial;
remaining = remaining - 3;
final int CAPACITY = 20;
System.out.println(remaining);
System.out.println(initial);
```

Expected result and interpretation:

```
7
10
Assigning remaining does not change initial.
```

Guided problem and trace

Read a quantity and price using Scanner. Print their product. Then read a one-letter category. Use quantity 3, price 12.5, category A.

1. Create one Scanner and read quantity, price and category in order.

2. Calculate quantity * price using the numeric values.

3. Print the product and the selected category.

Input:

```
3 12.5 A
```

```
java.util.Scanner in = new java.util.Scanner(System.in);
in.useLocale(java.util.Locale.US);
int quantity = in.nextInt();
double price = in.nextDouble();
char category = in.next().charAt(0);
double total = quantity * price;
System.out.println(total);
System.out.println(category);
```

Read or calculation	Consumed token	Stored result
nextInt()	3	quantity = 3
nextDouble()	12.5	price = 12.5
next().charAt(0)	A	category = 'A'
quantity * price	No new input	total = 37.5

Expected output:

```
37.5
A
```

Input tokens become typed values

Each Scanner call consumes the next token according to the requested type.

Relevant labels: 12 2.5 A; nextInt: 12; nextDouble: 2.5; next: A

Next token	Requested operation	Result
12	nextInt()	int value 12
2.5	nextDouble()	double value 2.5
A	next().charAt(0)	char value A

Additional worked example: Circle area with console input

Declaration introduces radius and area; assignment gives a local variable a usable value. A named constant documents the chosen approximation to pi. Read the radius as double, then calculate only after input is available.

Read radius 20 and calculate its area with PI = 3.14159, matching the source example. Assume a nonnegative radius.

Input: 20

```
java.util.Scanner input = new java.util.Scanner(System.in);
input.useLocale(java.util.Locale.US);
final double PI = 3.14159;
double radius = input.nextDouble();
double area = PI * radius * radius;
System.out.printf(java.util.Locale.US, "Area: %.3f%n", area);
```

Moment	radius	area
After reading	20.0	Not assigned yet
After calculation	20.0	1256.636
Radius changed to zero	0.0	Recalculation gives 0.000

Expected output:

```
Area: 1256.636
```

Independent practice

1. A shop sells 12 items at 2.5 each. Show the variable declarations and total. Then predict what happens if nextInt() is used for the token 2.5.

2. Why does changing radius after calculating area not automatically change area?

3. Debugging: int total;
System.out.println(total);

4. Check your understanding: After int x=4; int y=x; y=8;, what is x? Can a final variable be assigned again?

5. Homework: Read a rectangle length and width. Print area and perimeter. State the units and allowed input range.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_04_Variables_and_console_input.tex and PDF/Lecture_04_Variables_and_console_input.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture04.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-5.pptx, slides 3, 4, 5, 6, 12, 15, 22, 23. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 05: Types and arithmetic expressions

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Liang Ch 2. Learning outcome: CLO-1.

Learning objectives

- Choose suitable primitive types

- Evaluate precedence and integer division

- Use casts and conversions deliberately

Concepts explained

Primitive data types

Integer types: byte, short, int and long. Floating-point types: float and double. Other primitives: char and boolean. A String is a reference type.

```
int count = 20;
long population = 8000000000L;
double average = 72.5;
boolean passed = true;
```

Division and remainder

Integer division truncates toward zero. The % operator returns a remainder. Its sign follows the dividend. At least one floating-point operand gives floating-point division.

```
7 / 2       // 3
7 / 2.0     // 3.5
7 % 2       // 1
-7 / 2      // -3
```

Precedence and grouping

Multiplication, division and remainder bind before addition and subtraction. Operators at these arithmetic levels associate left to right. Parentheses make the intended calculation explicit.

```
2 + 3 * 4       // 14
(2 + 3) * 4     // 20
20 / 5 * 2      // 8
```

Conversions and numeric limits

Widening conversions often happen automatically. A narrowing cast can discard a fraction or lose range. Floating-point values approximate many decimal fractions.

```
double x = 7;
int y = (int) 7.9;  // 7
double mean = (double) 7 / 2;
```

Quotient and remainder decompose a quantity

Whole-unit conversion uses integer division to count complete larger units. Remainder keeps the part that has not yet been converted. Convert the remainder again when more than two units are needed.

```
3672 seconds
1 complete hour leaves 72 seconds
1 complete minute leaves 12 seconds
```

Casting changes a value at a specific point

A cast affects the expression at the location where it occurs. Casting an already truncated quotient cannot restore its discarded fraction. Promote an operand before division when a fractional result is required.

```
(double) (7 / 2) gives 3.0
(double) 7 / 2 gives 3.5
```

Key distinctions

Expression	Operand arithmetic	Result
7 / 2	Integer division	3
7 / 2.0	Floating-point division	3.5
7 % 2	Integer remainder	1
(int) 7.9	Narrowing conversion	7
(double) (7 / 2)	Cast after division	3.0

First worked example

```
int total = 145;
int count = 2;
double wrong = total / count;
double correct = (double) total / count;
System.out.println(wrong);
System.out.println(correct);
```

Expected result and interpretation:

```
72.0
72.5
The first expression performs integer division before assignment.
The cast changes an operand before division.
```

Guided problem and trace

Convert 3672 seconds to hours, minutes and seconds using / and %.

1. Find complete hours with seconds / 3600.

2. Keep seconds % 3600 for the part after complete hours.

3. Divide that remainder by 60, then use % 60 for remaining seconds.

```
int totalSeconds = 3672;
int hours = totalSeconds / 3600;
int remainder = totalSeconds % 3600;
int minutes = remainder / 60;
int seconds = remainder % 60;
System.out.println(hours + ":" + minutes + ":" + seconds);
```

Variable	Calculation	Value
hours	3672 / 3600	1
remainder	3672 % 3600	72
minutes	72 / 60	1
seconds	72 % 60	12

Expected output:

```
1:1:12
```

Arithmetic follows the expression tree

The denominator 2.0 makes the final division floating-point.

Relevant labels: /; +; 2.0; 7; 8

Expression	Result	Reason
(7 + 8) / 2	7	Integer quotient
(7 + 8) / 2.0	7.5	Floating-point divisor
(double) ((7 + 8) / 2)	7.0	The fraction was already lost

Additional worked example: Division and compound assignment

Integer division discards the fractional part; a floating-point operand changes the arithmetic. A compound assignment includes conversion back to the left variable type. Therefore x /= 3.0 for an int x is not equivalent to the uncompilable x = x / 3.0.

Compare 5/2, 5.0/2 and int x=10 followed by x/=3.0. Explain each result.

Input: Fixed values in the example.

```
System.out.println(5 / 2);
System.out.println(5.0 / 2);
int x = 10;
x /= 3.0;
System.out.println(x);
```

Expression	Arithmetic before storage	Result
5 / 2	Integer	2
5.0 / 2	double	2.5
x /= 3.0, x initially 10	double then int conversion	3

Expected output:

```
2
2.5
```

3

Independent practice

1. Convert 5,439 seconds to hours, minutes and seconds. Explain why quotient and remainder are both needed.

2. Rewrite x /= 3.0 with an explicit assignment that compiles and preserves the demonstrated conversion.

3. Debugging: double average = (double) (7 / 2);

4. Check your understanding: Evaluate 10 - 3 * 2 and (10 - 3) * 2. What does (int) -2.9 produce?

5. Homework: Write a temperature converter. Test 0 and 100 Celsius. Explain why 9 / 5 gives the wrong conversion factor.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_05_Types_and_arithmetic_expressions.tex and PDF/Lecture_05_Types_and_arithmetic_expressions.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture05.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-4.pptx, slides 13, 16, 17. Clarified the implicit conversion in compound assignment, which the source equivalence omits.

Lecture 06: Updates and formatted output

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Liang Ch 2. Learning outcome: CLO-1.

Learning objectives

- Trace prefix and postfix updates

- Distinguish print, println and printf

- Format a readable numeric report

Concepts explained

Increment and decrement

x++ and ++x both increase x by one. Postfix produces the old value. Prefix produces the updated value. Standalone updates are easier to read.

```
int x = 4;
int a = x++; // a=4, x=5
int b = ++x; // x=6, b=6
```

Compound assignment

x += amount updates x using its current value. The same pattern supports -=, *=, /= and %. Use ordinary assignment when a conversion needs explanation.

```
int stock = 20;
stock -= 3;
stock += 5;
// stock is 22
```

Output methods

print leaves the cursor on the current line. println ends the line. printf uses a format string and arguments. %n inserts a platform-appropriate line separator.

```
System.out.print("Total: ");
System.out.println(24);
System.out.printf("%d%n", 24);
```

Format specifiers

%d formats an integer and %s formats text. %.2f shows two decimal places for floating-point output. A width reserves a minimum field size.

```
System.out.printf("%-8s %6.2f%n",
                  "Tea", 125.5);
```

A stored value and an expression result differ

Postfix increment first produces the old value, then updates the variable. Prefix increment first updates the variable, then produces its new value. Separating updates from larger expressions makes this ordering easier to inspect.

```
int x = 4;
int old = x++;
// old is 4, x is 5
```

Formatted output as a small report

A format string defines how each supplied value appears. Field width is a minimum width, so a longer value can still expand the field. A consistent decimal format lets a reader compare prices quickly.

```
%-8s means left-aligned text in at least 8 positions.
%8.2f means a number with two decimal places.
```

Key distinctions

Format	Argument example	Displayed purpose
%d	25	Whole number
%.2f	25.5	25.50
%-8s	Pen	Left-aligned item name
%n	No argument	Platform line ending

First worked example

```
int count = 4;
int before = count++;
System.out.printf(java.util.Locale.US,
    "%d %d %.2f%n", before, count, 12.5);
```

Expected result and interpretation:

```
4 5 12.50
before receives 4, then count becomes 5.
The floating-point value displays two decimal places.
```

Guided problem and trace

Print a two-column receipt for Pen 25.5 and Book 120.0. Use aligned names and two decimal places.

1. Print each item with a fixed-width name and a numeric price field.

2. Use Locale.US to keep the demonstrated decimal point consistent.

3. Calculate the total separately, then format it with the same price pattern.

```
double pen = 25.5;
double book = 120.0;
System.out.printf(java.util.Locale.US, "%-8s %8.2f%n", "Pen", pen);
System.out.printf(java.util.Locale.US, "%-8s %8.2f%n", "Book", book);
System.out.printf(java.util.Locale.US, "%-8s %8.2f%n", "Total", pen + book);
```

Item	Stored price	Displayed price
Pen	25.5	25.50

Item	Stored price	Displayed price
Book	120.0	120.00
Total	145.5	145.50

Expected output:

```
Pen         25.50
Book       120.00
Total      145.50
```

Postfix preserves the old expression value

Separate the expression result from the variable state after evaluation.

Relevant labels: x starts at 4; y = x++; y receives 4; x becomes 5

Statement (fresh x=4)	Assigned y	Final x
y = x++	4	5
y = ++x	5	5
x++; y = x	5	5

Additional worked example: Tracing four update operators

Java evaluates these operands left to right, so each update affects later operands. Record the value supplied by an operand separately from the new value of i. Such expressions are useful trace exercises; separate updates are easier to maintain.

Trace int i=5 and result=++i+i--+i+++--i, written with spaces in the program. Predict both printed values.

Input: Fixed values in the example.

```
int i = 5;
int result = ++i + i-- + i++ + --i;
System.out.println("Result: " + result);
System.out.println("Final i: " + i);
```

Operand	Value supplied	i afterward
++i	6	6
i--	6	5
i++	5	6
--i	5	5

Expected output:

```
Result: 22
Final i: 5
```

Independent practice

1. Trace x=4, y=x++, z=++x. Then print y, z and x in labelled columns.

2. Replace the expression with four named intermediate values and verify the same total.

3. Debugging: System.out.printf("Price: %d", 12.5);

4. Check your understanding: With x=2, what does y=++x do? Does %.2f round the stored variable?

5. Homework: Create a marks report showing a name, integer marks and percentage to one decimal place.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_06_Updates_and_formatted_output.tex and PDF/Lecture_06_Updates_and_formatted_output.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture06.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-6.pptx, slides 6, 7, 15, 16, 17. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 07: Boolean expressions

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 7 (as listed). Learning outcome: CLO-2.

Learning objectives

- Construct relational and logical conditions

- Evaluate Boolean expressions

- Choose explicit grouping for compound rules

Concepts explained

Relational operators

<, <=, > and >= compare numeric values. == tests primitive equality and != tests inequality. A relational expression produces true or false.

```
int marks = 50;
boolean passed = marks >= 50;
boolean perfect = marks == 100;
```

Logical combinations

&& requires both conditions to be true. || requires at least one condition to be true. ! reverses a Boolean value.

```
boolean eligible = age >= 18 && registered;
boolean weekend = day == 6 || day == 7;
```

Intervals and precedence

Write a range check as two comparisons joined with &&. Relational tests bind before &&, which binds before ||. Parentheses make mixed rules easier to inspect.

```
boolean valid = marks >= 0 && marks <= 100;
boolean accepted = (a || b) && c;
```

Negation and boundary cases

The opposite of x < limit is x >= limit. De Morgan: !(a && b) equals !a || !b. Tests at the boundary reveal missing equality cases.

```
!(age >= 18 && registered)

Equivalent:
age < 18 || !registered
```

Translating a sentence into a condition

"Below 12" uses < 12, while "at least 60" uses >= 60. The word "or" permits either qualifying group. Test the endpoints to confirm that the inequalities reflect the wording.

```
Discount rule:
age < 12 || age >= 60
```

A Boolean expression is a value

The result of a comparison can be stored, passed or combined. A condition must ultimately produce boolean in Java. A number such as 1 is not a replacement for true.

```
boolean discounted = age < 12 || age >= 60;
System.out.println(discounted);
```

Key distinctions

A	B	A && B	A || B
false	false	false	false
false	true	false	true
true	false	false	true
true	true	true	true

First worked example

```
int marks = 50;
boolean valid = marks >= 0 && marks <= 100;
boolean passed = valid && marks >= 50;
System.out.println(valid);
System.out.println(passed);
```

Expected result and interpretation:

```
true
true
50 satisfies the range and includes the passing boundary.
```

Guided problem and trace

A ticket discount applies to ages below 12 or at least 60. Write the condition and test ages 11, 12, 59 and 60.

1. Express each qualifying age group as its own comparison.

2. Join the comparisons with ||.

3. Evaluate ages immediately below and at each boundary.

```
int age = 11;
System.out.println(age < 12 || age >= 60);
age = 12;
System.out.println(age < 12 || age >= 60);
age = 59;
System.out.println(age < 12 || age >= 60);
age = 60;
System.out.println(age < 12 || age >= 60);
```

Age	age < 12	age >= 60	Discount
11	true	false	true

Age	age < 12	age >= 60	Discount
12	false	false	false
59	false	false	false
60	false	true	true

Expected output:

```
true
false
false
true
```

Build a compound condition from two facts

Admission requires both facts to be true. One true input is insufficient.

Relevant labels: age >= 18; hasCard; AND; admitted

Age	Has card	age >= 18 && hasCard
17	true	false
18	false	false
18	true	true

Additional worked example: Divisibility with AND, OR and XOR

Remainder zero expresses divisibility by a nonzero integer. AND requires both properties; OR accepts either or both. Boolean XOR is true when exactly one property is true.

For 6, evaluate divisibility by 2 and 3 using all three Boolean combinations.

Input: Fixed values in the example.

```
int number = 6;
boolean by2 = number % 2 == 0;
boolean by3 = number % 3 == 0;
System.out.println(by2 && by3);
System.out.println(by2 || by3);
System.out.println(by2 ^ by3);
```

Number	AND / OR	XOR
6	true / true	false
4	false / true	true
5	false / false	false

Expected output:

```
true
true
```

```
false
```

Independent practice

1. Admission requires age at least 18 and a valid card. Write both an admitted condition and its logical negation. Test the age boundary.

2. Why would independent if statements be appropriate when displaying all properties that hold?

3. Debugging: boolean inRange = 0 <= marks <= 100;

4. Check your understanding: Evaluate true || false && false. What is the opposite of score >= 50?

5. Homework: Write a library eligibility condition from two age and membership rules of your choice. Supply a truth table.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_07_Boolean_expressions.tex and PDF/Lecture_07_Boolean_expressions.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture07.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-8-9.pptx, slides 31, 32, 33, 34, 35. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 08: Selection with if and else

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 7 (as listed). Learning outcome: CLO-2.

Learning objectives

- Implement one-way and two-way selection

- Build an ordered multi-branch decision

- Test boundaries and nested conditions

Concepts explained

The if statement

An if condition must be Boolean. The body executes only when the condition is true. Braces clearly group all statements in the branch.

```
if (balance < 0) {
  System.out.println("Overdrawn");
}
```

Two-way selection

An if-else chooses exactly one branch. Statements after the structure execute after either branch completes normally.

```
if (marks >= 50) {
  System.out.println("Pass");
} else {
  System.out.println("Fail");
}
```

Multiple selections

An else-if chain selects the first matching branch. Order overlapping thresholds from most restrictive to least restrictive. Independent if statements can execute several bodies.

```
if (score >= 80) {
  band = "High";
} else if (score >= 50) {
  band = "Middle";
} else { band = "Low"; }
```

Nesting and compound statements

A nested if expresses a decision inside another decision. An else attaches to the nearest unmatched if. Braces prevent misleading indentation.

```
if (registered) {
  if (paid) {
    System.out.println("Admit");
  }
}
```

Branch order determines the selected result

An else-if chain stops testing after its first matching condition. Overlapping ranges need an order that gives the most specific case priority. A final else covers every case not selected earlier.

```
If >=50 comes before >=80, a score of 90
selects the >=50 branch immediately.
```

Independent tests and exclusive alternatives

Separate if statements can execute more than one body. An if-else chain expresses mutually exclusive outcomes. For sign classification, exactly one of negative, zero and positive is appropriate.

```
if (n < 0) ...
else if (n == 0) ...
else ...
```

Key distinctions

Input n	n < 0	n == 0 if reached	Chosen result
-1	true	Not evaluated	Negative
0	false	true	Zero
1	false	false	Positive

First worked example

```
int score = 80;
String band;
if (score >= 80) { band = "High"; }
else if (score >= 50) { band = "Middle"; }
else { band = "Low"; }
System.out.println(band);
```

Expected result and interpretation:

```
High
The first condition is true.
The remaining branches are skipped.
```

Guided problem and trace

Classify an integer as negative, zero or positive. Draw a decision outline, then write an if-else chain.

1. Check the negative case first.

2. If the number is not negative, test equality to zero.

3. Every remaining integer is positive.

```
int n = 0;
if (n < 0) {
  System.out.println("Negative");
} else if (n == 0) {
  System.out.println("Zero");
} else {
  System.out.println("Positive");
}
```

Execution point	Condition or action	Result
First test	0 < 0	false
Second test	0 == 0	true
Selected body	Print Zero	Zero
Remaining branch	Skip else	No extra output

Expected output:

```
Zero
```

A decision selects one path

Exactly one branch executes; both normal paths rejoin after the selection.

Relevant labels: mark >= 50?; Print Pass; Print Fail; Continue

Mark	Condition	Printed result
49	false	Fail
50	true	Pass
51	true	Pass

Additional worked example: Overtime payroll

The first 40 hours use the regular rate. Only excess hours use the 1.5 multiplier. At exactly 40 hours, overtime hours are zero. This is the fictional business rule supplied by the exercise, not an employment-law rule.

Compute pay for 45 hours at 20 units per hour using the source overtime rule.

Input: Fixed values in the example.

```
double hours = 45, rate = 20;
double pay;
if (hours > 40) {
  pay = 40 * rate + (hours - 40) * rate * 1.5;
} else {
  pay = hours * rate;
}
System.out.printf(java.util.Locale.US, "Pay: %.2f%n", pay);
```

Hours	Regular + overtime pay	Total
39	780 + 0	780
40	800 + 0	800
45	800 + 150	950

Expected output:

```
Pay: 950.00
```

Independent practice

1. Write a grade band selection: 80 or above is High, 50 through 79 is Pass, and below 50 is Fail. Explain the order of tests.

2. What is wrong with multiplying all 45 hours by 1.5 times the rate?

3. Debugging: if (marks >= 50);
{ System.out.println("Pass"); }

4. Check your understanding: How many branches can an else-if chain select? Why test 49, 50 and 51?

5. Homework: Write a shipping-charge decision for three weight ranges. State your rules and test each boundary.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_08_Selection_with_if_and_else.tex and PDF/Lecture_08_Selection_with_if_and_else.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture08.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-7.pptx, slides 24, 25. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 09: Short-circuiting and switch

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 7 (as listed). Learning outcome: CLO-2.

Learning objectives

- Use short-circuit guards safely

- Choose a conditional expression

- Implement a classic switch statement

Concepts explained

Short-circuit evaluation

&& skips its right operand when the left operand is false. || skips its right operand when the left operand is true. A guard must precede the operation it protects.

```
boolean safe = divisor != 0
    && numerator / divisor > 2;
```

Conditional operator

condition ? valueIfTrue : valueIfFalse produces a value. Use it for short alternatives with compatible types. Use if-else for longer branch behaviour.

```
int larger = a >= b ? a : b;
```

Classic switch selection

switch selects a branch from a selector value. case labels are constant alternatives. default handles unmatched values. Classic colon cases continue into later cases unless control exits.

```
switch (choice) {
  case 1: label = "Open"; break;
  case 2: label = "Save"; break;
  default: label = "Unknown";
}
```

Choice of selection structure

if-else handles ranges and compound conditions. switch suits discrete alternatives such as menu choices. Document any intentional fall-through.

```
Range: temperature < 0
Discrete choice: menu option 1, 2 or 3
```

A guard protects only later evaluation

Short-circuiting follows left-to-right evaluation. A safe condition first establishes the requirement for the next operation. With integer division, the divisor check must precede the division.

```
d != 0 && 10 / d > 1
For d=0, the right side never runs.
```

Switch fall-through is an execution rule

A matching classic case determines where execution enters the switch. It does not automatically determine where execution leaves. break transfers control to the statement after the switch.

```
case 1: print Add; break;
case 2: print Remove; break;
default: print Invalid;
```

Key distinctions

Construct	Best suited to	Key behaviour
if-else	Ranges and compound rules	First matching branch
?:	Short value alternatives	Produces a selected value
Classic switch	Discrete menu alternatives	Falls through without an exit

First worked example

```
int choice = 2;
String label;
switch (choice) {
  case 1: label = "Open"; break;
  case 2: label = "Save"; break;
  default: label = "Unknown";
}
System.out.println(label);
```

Expected result and interpretation:

```
Save
Execution enters case 2.
break exits the switch.
```

Guided problem and trace

Write a menu mapping 1 to Add, 2 to Remove and every other integer to Invalid. Test 1, 2 and 9.

1. Read one integer menu choice.

2. Use distinct case labels for 1 and 2, with a break in each.

3. Use default to report every unmatched integer.

Input:

```
2
```

```
java.util.Scanner in = new java.util.Scanner(System.in);
int choice = in.nextInt();
switch (choice) {
  case 1:
    System.out.println("Add");
    break;
  case 2:
    System.out.println("Remove");
    break;
  default:
```

```
    System.out.println("Invalid");
}
```

Choice	Matching case	Output	Exit
1	case 1	Add	break
2	case 2	Remove	break
9	default	Invalid	End of switch

Expected output:

```
Remove
```

A short-circuit guard controls evaluation

When d is zero, Java skips the division and the entire AND expression is false.

Relevant labels: d != 0?; Evaluate 10 / d > 1; Result false; Use Boolean result

d	Right operand evaluated?	Result
0	No	false
2	Yes: 5 > 1	true
10	Yes: 1 > 1	false

Additional worked example: Letter grades as a switch problem

A fixed mapping from one letter to one value fits discrete selection. Normalise case once, then reject letters outside the specified grade set. In classic switch syntax, break prevents unintended fall-through.

Map A, B, C, D and F to 4, 3, 2, 1 and 0. Demonstrate lowercase b after case normalisation.

Input: Fixed values in the example.

```
char grade = Character.toUpperCase('b');
int points;
switch (grade) {
  case 'A': points = 4; break;
  case 'B': points = 3; break;
  case 'C': points = 2; break;
  case 'D': points = 1; break;
  case 'F': points = 0; break;
  default: points = -1;
}
System.out.println(points);
```

Input	Normalised	Result
b	B	3
F	F	0
E	E	-1: invalid grade

Expected output:

```
3
```

Independent practice

1. Explain why reversing the operands in d != 0 && 10 / d > 1 is unsafe. Write a guarded test and predict it for d=0, 2 and 10.

2. If input is a String rather than a char, what must be checked before charAt(0)?

3. Debugging: boolean ok = 10 / d > 1 && d != 0;

4. Check your understanding: What does false && riskyCall() do? Why can a missing break change output?

5. Homework: Create a four-option calculator using switch. Handle an invalid option and guard integer division by zero.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_09_Short_circuiting_and_switch.tex and PDF/Lecture_09_Short_circuiting_and_switch.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture09.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Strings -practice.pptx, slides 4, 5; Lecture-8-9.pptx, slides 16, 17, 18, 20. Re-expressed the source if-else grade mapping as a classic switch to match this lecture.

Source exercise deck credits Instructor Khurram Iqbal.

Lecture 10: While loops

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 8 (as listed). Learning outcome: CLO-2.

Learning objectives

- Design terminating while loops

- Use counter, sentinel and flag control

- Trace state across iterations

Concepts explained

The while structure

A while loop checks its condition before each iteration. Its body may run zero times. Initialisation, condition and progress must agree.

```
int i = 1;
while (i <= 3) {
  System.out.println(i);
  i++;
}
```

Counter-controlled repetition

Use a counter when the number of repetitions is known. An accumulator starts at an appropriate identity, such as zero for a sum. State the range clearly.

```
int total = 0;
int i = 1;
while (i <= 4) {
  total += i;
  i++;
}
```

Sentinel-controlled repetition

A sentinel marks the end of input and is not ordinary data. Read before the loop and again after processing each accepted value. Do not include the sentinel in a sum or count.

```
int x = in.nextInt();
while (x != -1) {
  total += x;
  count++;
  x = in.nextInt();
}
```

Flag-controlled repetition

A Boolean flag can record whether a condition has been achieved. Every path must permit progress toward stopping. A loop that never changes its condition may not terminate.

```
boolean found = false;
```

```
int candidate = 1;
while (!found && candidate <= 10) {
  found = candidate % 7 == 0;
  candidate++;
}
```

A loop maintains a useful relationship

After processing k marks, count should equal k and sum should contain their total. The next iteration extends this relationship with one accepted mark. This makes the final calculation easier to justify.

```
Before input: count=0, sum=0
After 60: count=1, sum=60
After 80: count=2, sum=140
```

The sentinel is control data

The stop value ends repetition and contributes neither a mark nor a count. Checking before accumulation prevents a biased sum. An immediate sentinel leaves count at zero, so there is no average to divide for.

```
Input: 60 80 -1
Data values: 60 and 80
Sentinel: -1
```

Key distinctions

Control style	Stop rule	Typical use
Counter	Required number processed	Read exactly 5 marks
Sentinel	Special input encountered	Read until -1
Flag	State becomes true or false	Stop after a match

First worked example

```
int i = 1;
int total = 0;
while (i <= 4) {
  total += i;
  i++;
}
System.out.println(total);
```

Expected result and interpretation:

```
10
After bodies: (i,total) = (2,1), (3,3), (4,6), (5,10).
The condition fails when i is 5.
```

Guided problem and trace

Design a sentinel loop for marks ending at -1. Print the count and average, or No data when the sentinel arrives first.

1. Initialise sum and count to zero, then read the first mark.

2. While the mark is not -1, accumulate it and read the next mark.

3. If count is positive, divide using floating-point arithmetic. Otherwise print No data.

Input:

```
60 80 -1
```

```
java.util.Scanner in = new java.util.Scanner(System.in);
int sum = 0;
int count = 0;
int mark = in.nextInt();
while (mark != -1) {
  sum += mark;
  count++;
  mark = in.nextInt();
}
if (count == 0) System.out.println("No data");
else System.out.println((double) sum / count);
```

Input	mark != -1	sum after body	count after body
60	true	60	1
80	true	140	2
-1	false	140 unchanged	2 unchanged

Expected output:

```
70.0
```

A while loop checks before every iteration

The back edge returns to the condition. A false first check gives zero iterations.

Relevant labels: Initialise i = 1; i <= 3?; Use i; increment; Continue after loop

Check	i	Action
First	1	Use 1; set i to 2
Third	3	Use 3; set i to 4
Final	4	Exit without using 4

Additional worked example: A bounded number-guessing loop

The flag records success; the counter limits how long the loop may continue. Increment the attempt count once for each accepted integer guess. Use a fixed secret while tracing so the expected outcome is reproducible.

Use secret 42 and at most five guesses. Test input 10, 80, 42 and report Low, High, then Correct.

Input: 10 80 42

```
java.util.Scanner in = new java.util.Scanner(System.in);
int secret = 42, attempts = 0;
boolean done = false;
while (attempts < 5 && !done) {
  int guess = in.nextInt();
  attempts++;
  if (guess == secret) {
    done = true;
```

```
    System.out.println("Correct");
  } else {
    System.out.println(guess < secret ? "Low" : "High");
  }
}
System.out.println("Attempts: " + attempts);
```

Guess	attempts	done / message
10	1	false / Low
80	2	false / High
42	3	true / Correct

Expected output:

```
Low
High
Correct
Attempts: 3
```

Independent practice

1. Write a while loop that adds the odd integers 1, 3, 5 and 7. State the final counter value and the number of condition checks.

2. Give an input sequence that exercises termination by the attempt limit rather than success.

3. Debugging: int i = 1;
while (i <= 5) { System.out.println(i); }

4. Check your understanding: Can while run zero times? Why must a sentinel be outside the normal input domain?

5. Homework: Use while to compute the digit sum of a nonnegative integer. Test 0, 7 and 204.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_10_While_loops.tex and PDF/Lecture_10_While_loops.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture10.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-10.pptx, slides 12, 13, 14, 22, 23, 24. Replaced randomness with secret 42 for reproducible tracing; numeric input is assumed. Kept the source five-attempt limit.

Lecture 11: For and do-while loops

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 8 (as listed). Learning outcome: CLO-2.

Learning objectives

- Select for, while or do-while appropriately

- Explain break and continue

- Trace nested iteration

Concepts explained

The for loop

A for header groups initialisation, condition and update. The update runs after each normally completed body. A header variable belongs to the loop scope.

```
for (int i = 0; i < 4; i++) {
  System.out.println(i);
}
```

The do-while loop

do-while checks its condition after the body. The body therefore runs at least once. The statement ends with a semicolon.

```
int n = 0;
do {
  System.out.println(n);
  n++;
} while (n < 0);
```

Break and continue

break exits the nearest loop. continue skips the rest of the current iteration. In a for loop, continue proceeds to the update step.

```
for (int i = 1; i <= 5; i++) {
  if (i == 2) continue;
  if (i == 4) break;
  System.out.println(i);
}
```

Nested loops

The inner loop completes for each outer iteration. With independent bounds, body executions multiply. Choose distinct names for separate counters.

```
for (int row = 1; row <= 2; row++) {
  for (int col = 1; col <= 3; col++) {
    System.out.print("*");
  }
  System.out.println();
```

```
}
```

The for update follows continue

A for loop transfers from continue to its update expression. A while loop transfers from continue directly to its condition. This difference matters when progress depends on a body statement that continue might skip.

```
for (...; ...; i++) {
  if (skip) continue;
}
The i++ update still runs.
```

Row boundaries belong to the outer loop

The inner loop prints all characters in one row. A newline after the inner loop ends that row. Putting the newline inside the inner loop would produce one character per line.

```
Row 1: #### then newline
Row 2: #### then newline
Row 3: #### then newline
```

Key distinctions

Loop	Condition check	Minimum body executions
while	Before body	0
for	Before body	0
do-while	After body	1

First worked example

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
  if (i == 3) continue;
  sum += i;
}
System.out.println(sum);
```

Expected result and interpretation:

```
12
The added values are 1, 2, 4 and 5.
continue still allows the for update to run.
```

Guided problem and trace

Print a 3 by 4 rectangle of # characters using nested loops. Explain how many times the inner body runs.

1. Repeat three times for the three rows.

2. Inside each row, print four # characters without ending the line.

3. Print one newline after completing the inner loop.

```
for (int row = 0; row < 3; row++) {
  for (int column = 0; column < 4; column++) {
    System.out.print("#");
  }
```

```
  System.out.println();
}
```

Outer row	Inner column values	Printed content	Inner executions
0	0, 1, 2, 3	####	4
1	0, 1, 2, 3	####	4
2	0, 1, 2, 3	####	4
All rows	3 rows x 4 columns	Three lines	12

Expected output:

```
####
####
####
```

Nested loops visit a rectangular grid

The inner loop completes a row before the outer loop moves to the next row.

Relevant labels: r0c0; r0c1; r0c2; r1c0; r1c1; r1c2

Outer index	Inner indexes	Body executions
0	0, 1, 2	3
1	0, 1, 2	3
Total	2 rows x 3 columns	6

Additional worked example: A multiplication table with nested loops

The row counter supplies one factor and the column counter supplies the other. Reset the column loop for each new row. Formatting belongs inside the inner loop; the newline belongs after it.

Print the 1-through-3 multiplication table using nested for loops.

Input: Fixed values in the example.

```
for (int row = 1; row <= 3; row++) {
  for (int col = 1; col <= 3; col++) {
    System.out.printf("%3d", row * col);
  }
  System.out.println();
}
```

Row factor	Column factors	Printed products
1	1, 2, 3	1, 2, 3
2	1, 2, 3	2, 4, 6
3	1, 2, 3	3, 6, 9

Expected output:

```
1   2   3
2   4   6
3   6   9
```

Independent practice

1. Print two rows of three stars. Predict what changes if println() is moved inside the inner loop.

2. How many inner-body executions and newline calls occur for a 10-by-10 table?

3. Debugging: int i=0;
while(i<5) {
if(i==2) continue;
i++;
}

4. Check your understanding: Which loop guarantees one body execution? Does break exit every nested loop?

5. Homework: Print a multiplication table for 1 through 5. Use formatted output and nested loops.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_11_For_and_do_while_loops.tex and PDF/Lecture_11_For_and_do_while_loops.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture11.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-11.pptx, slides 14, 15. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 12: References and string input

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 3. Learning outcome: CLO-2.

Learning objectives

- Distinguish values from references

- Explain aliasing and null

- Read tokens and complete lines

Concepts explained

Primitive and reference variables

A primitive variable holds a primitive value. A reference variable holds a reference to an object or null. Copying a reference can make two variables refer to one object.

```
int count = 3;
String name = "Amina";
String sameName = name;
```

Objects and null

new requests construction of an object. null means a reference does not refer to an object. Calling an instance method through null throws NullPointerException.

```
String text = null;
boolean missing = text == null;
```

String values are immutable

A String object represents text. Operations such as concatenation produce a result without changing the original String object. Reassignment changes which result a variable refers to.

```
String a = "CS";
String b = a;
a = a + "C103";
// b still refers to "CS"
```

Token input and line input

next reads a token separated by whitespace. nextLine reads the rest of the current line. After nextInt, the pending line ending can yield an empty nextLine result.

```
int age = in.nextInt();
in.nextLine(); // finish the numeric line
String fullName = in.nextLine();
```

A token read leaves the line ending

nextInt consumes the integer token but usually leaves its following line separator. The next nextLine reads the remainder of that same line, which may be empty. When the full name belongs on the next line, finish the numeric line first.

```
Input:
20
Amina Khan

The first nextLine after nextInt finishes line 1.
```

Aliasing differs from reassignment

Two reference variables can identify the same object. Reassigning one variable changes only that variable's stored reference. String immutability means a transformation returns text rather than editing the original String.

```
String a = "CS";
String b = a;
a = "Math";
// b still denotes "CS"
```

Key distinctions

Value	Meaning	Safe length call?
null	No String object	No
""	An empty String	Yes, returns 0
"Ali"	A String with basic Latin text	Yes, returns 3

First worked example

```
String first = "CS";
String second = first;
first = first + "C103";
System.out.println(first);
System.out.println(second);
```

Expected result and interpretation:

```
CSC103
CS
The second variable still refers to the original text.
```

Guided problem and trace

Read a student age from one line and full name from the next. Explain why next() is insufficient for a name containing spaces.

1. Read the age as an integer token.

2. Consume the remainder of that line.

3. Read the next complete line as the name, preserving spaces.

Input:

```
20
Amina Khan
```

```
java.util.Scanner in = new java.util.Scanner(System.in);
int age = in.nextInt();
in.nextLine();
String fullName = in.nextLine();
System.out.println(fullName);
System.out.println(age);
```

Operation	What it reads	Stored result
nextInt()	20	age = 20
first nextLine()	Remainder of numeric line	Discarded empty text
second nextLine()	Amina Khan	fullName includes the space

Expected output:

```
Amina Khan
20
```

Two variables can refer to the same string

Before concatenation both references identify CS. Reassigning first leaves second unchanged.

Relevant labels: first; second; String object: CS; first + 103 gives a new result

Step	first refers to	second refers to
second = first	CS	CS
first = first + "103"	CSC103	CS
Print second	CSC103	CS is printed

Additional worked example: Ordering two city names

Read full lines so a city name may contain spaces. compareTo returns a negative value, zero, or a positive value; its sign is what matters. This comparison is case-sensitive lexicographic ordering, not locale-aware dictionary collation.

Read Islamabad and Sahiwal on separate lines and print them in lexicographic order.

Input: Islamabad
Sahiwal

```
java.util.Scanner in = new java.util.Scanner(System.in);
String first = in.nextLine();
String second = in.nextLine();
if (first.compareTo(second) <= 0) {
  System.out.println(first + " | " + second);
} else {
  System.out.println(second + " | " + first);
}
```

First / second	Comparison sign	Printed order
Islamabad / Sahiwal	Negative	Islamabad, Sahiwal
Sahiwal / Islamabad	Positive	Islamabad, Sahiwal

First / second	Comparison sign	Printed order
Sahiwal / Sahiwal	Zero	Both unchanged

Expected output:

```
Islamabad | Sahiwal
```

Independent practice

1. Predict the output after assigning a="PF", b=a, and a=a+"1". Explain why b does not change.

2. Why is testing compareTo(...) == -1 an incorrect general ordering test?

3. Debugging: String name = null;
System.out.println(name.length());

4. Check your understanding: Is "" the same as null? What happens to b after a="new" when b previously received a?

5. Homework: Build a profile input program with an integer ID and a full name on separate lines. Test a name with two spaces.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_12_References_and_string_input.tex and PDF/Lecture_12_References_and_string_input.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture12.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-12.pptx, slides 27, 28, 29, 30, 31. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 13: String processing

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 16. Learning outcome: CLO-2.

Learning objectives

- Use common String methods

- Compare text content and extract substrings

- Convert numeric text and numeric values

Concepts explained

Length and character access

length() counts UTF-16 code units. charAt(index) uses positions from 0 to length()-1. Basic Latin examples let each char correspond to one visible character.

```
String s = "Java";
s.length()    // 4
s.charAt(0)   // J
```

Content comparison

equals compares String content. equalsIgnoreCase ignores case distinctions for its comparison. compareTo returns a negative value, zero or a positive value.

```
"cat".equals("cat")        // true
"Cat".equals("cat")        // false
"cat".compareTo("dog") < 0 // true
```

Substrings and searching

substring(begin,end) includes begin and excludes end. indexOf returns the first matching position or -1. trim returns a result without leading or trailing basic whitespace.

```
String code = "CS-103";
code.substring(3)     // "103"
code.substring(0, 2)  // "CS"
code.indexOf("-")     // 2
```

Numbers and text

Integer.parseInt and Double.parseDouble parse numeric text. String.valueOf converts a value to text. Malformed numeric text raises NumberFormatException.

```
int n = Integer.parseInt("42");
String label = String.valueOf(n);
```

Substring boundaries form a half-open interval

A substring starts at begin and stops before end. The difference end - begin gives its UTF-16 length. An empty substring is valid when begin equals end.

```
"JAVA".substring(1,3) is "AV"
Included indexes: 1 and 2
Excluded index: 3
```

Search failure must be handled before slicing

indexOf returns -1 when it finds no match. Passing -1 as a substring boundary is invalid. Validate the position before extracting the two parts of a structured value.

```
For "ali@campus.edu": position=3
Username: substring(0,3)
Domain: substring(4)
```

Key distinctions

Operation	Question answered	Example result
equals	Same text content?	"CS" equals "CS": true
==	Same object reference?	May differ for equal text
indexOf	Where is the first match?	"CS-103" finds "-" at 2
substring	Which interval of text?	substring(3) gives "103"

First worked example

```
String code = "CS-103";
String prefix = code.substring(0, 2);
int number = Integer.parseInt(code.substring(3));
System.out.println(prefix);
System.out.println(number + 1);
```

Expected result and interpretation:

```
CS
104
The numeric substring is "103". Parsing enables arithmetic.
```

Guided problem and trace

For a nonempty email-like string with one @, extract the username and domain. Test ali@campus.edu.

1. Find the @ position and reject missing or empty parts.

2. Extract text before @ as the username.

3. Extract text after @ as the domain.

```
String email = "ali@campus.edu";
int at = email.indexOf("@");
if (at <= 0 || at == email.length() - 1) {
  System.out.println("Invalid format");
} else {
  String user = email.substring(0, at);
  String domain = email.substring(at + 1);
  System.out.println(user);
  System.out.println(domain);
}
```

Part	Index interval	Result
Username	[0, 3)	ali
Separator	index 3	@
Domain	[4, 14)	campus.edu

Expected output:

```
ali
campus.edu
```

Substring boundaries sit between characters

substring(0,3) selects positions 0, 1 and 2. The end index 3 is excluded.

Relevant labels: C; S; C; 1; 0; 3

Call on "CSC103"	Indexes used	Result
substring(0,3)	0, 1, 2	CSC
substring(3)	3, 4, 5	103
substring(6)	No characters	Empty string

Additional worked example: Counting vowels in a sentence

Scan each character once and increase the count only on a vowel match. Normalise case to make uppercase and lowercase English vowels equivalent. This exercise counts a, e, i, o and u; it is not a general linguistic vowel detector.

Count the vowels in Welcome to Java. Predict the six matching characters before running.

Input: Fixed values in the example.

```
String text = "Welcome to Java".toLowerCase(java.util.Locale.ROOT);
int count = 0;
for (int i = 0; i < text.length(); i++) {
  char ch = text.charAt(i);
  if ("aeiou".indexOf(ch) >= 0) count++;
}
System.out.println(count);
```

Word	Matching vowels	Cumulative count
Welcome	e, o, e	3
to	o	4
Java	a, a	6

Expected output:

```
6
```

Independent practice

1. Given "lab:42", find the colon and convert the text after it to an int. State the assumptions needed for the conversion.

2. Predict the counts for an empty string, AEIOU and rhythm.

3. Debugging: String name = new String("Ali");
if (name == "Ali") { /* matched */ }

4. Check your understanding: What is "JAVA".substring(1,3)? Does s.trim() change the String referred to by s?

5. Homework: Read an item code shaped like BOOK-250. Extract the category and price, then print the price plus 10.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_13_String_processing.tex and PDF/Lecture_13_String_processing.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture13.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Strings -practice.pptx, slides 8, 9; Methods Practice.pptx, slides 10, 11. Uses Locale.ROOT and a compact membership check while retaining the source phrase and vowel-count task.

Source exercise deck credits Instructor Khurram Iqbal.

Lecture 14: Built-in and instance methods

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 6. Learning outcome: CLO-2.

Learning objectives

- Distinguish static and instance method calls

- Explain the conventional main method

- Use Math and Character APIs

Concepts explained

Methods as named operations

A method has a name, parameters and a result type or void. Arguments supply actual input values at a call. An API describes what callers may rely on.

```
Math.max(12, 9)
// name: max
// arguments: 12 and 9
// result: 12
```

Static and instance calls

A static method belongs to a class. An instance method operates through an object reference. Use the form that the API declares.

```
Math.sqrt(81.0)   // static
"Java".length()   // instance
```

The main entry point

The conventional main method is public and static, and returns void. String[] args receives command-line arguments. The JVM can call main without creating a class instance.

```
public static void main(String[] args) {
  System.out.println("Start");
}
```

Math and Character

Math provides operations such as abs, min, max, pow and sqrt. Character offers tests such as isDigit and isLetter. Check parameter types and exceptional cases in documentation.

```
Math.pow(2, 3)         // 8.0
Character.isDigit('7') // true
Character.toUpperCase('a') // A
```

Reading a method signature

The parameter list describes the kinds of inputs the method accepts. The result type describes the value that a caller receives. A static declaration permits a call through the class name.

```
Math.sqrt(double a) returns double.
An int argument can widen to double.
Math.sqrt(25) therefore gives 5.0.
```

Composing library calls

The result of one method call can supply an argument to another. Java evaluates the inner calls before the outer call receives their values. Intermediate variables can make a longer calculation easier to explain.

```
Math.max(Math.abs(-8), Math.abs(5))
becomes Math.max(8,5), which returns 8.
```

Key distinctions

Call	Kind	Input	Result
Math.abs(-8)	Static	int -8	int 8
Math.sqrt(25)	Static	number 25	double 5.0
"Java".length()	Instance	String receiver	int 4
Character.isDigit('7')	Static	char 7	boolean true

First worked example

```
double distance = Math.sqrt(3 * 3 + 4 * 4);
char initial = Character.toUpperCase('f');
System.out.println(distance);
System.out.println(initial);
```

Expected result and interpretation:

```
5.0
F
Math.sqrt returns a double. Character.toUpperCase returns a char here.
```

Guided problem and trace

Given x=-8 and y=5, print the larger absolute value using Math methods. Test whether character 7 is a digit.

1. Calculate the absolute value of each input.

2. Pass those results to Math.max.

3. Use Character.isDigit to classify the supplied character.

```
int x = -8;
int y = 5;
int absoluteX = Math.abs(x);
int absoluteY = Math.abs(y);
int larger = Math.max(absoluteX, absoluteY);
System.out.println(larger);
System.out.println(Character.isDigit('7'));
```

Call	Arguments supplied	Returned value
abs	-8	8
abs	5	5
max	8 and 5	8
isDigit	'7'	true

Expected output:

```
8
true
```

Nested method calls pass results outward

An inner call finishes before its returned value is used by the outer call.

Relevant labels: Math.abs(-6); 6; Math.max(6,4); 6

Call	Argument meaning	Result
Math.abs(-6)	Magnitude of -6	6
Math.max(6,4)	Larger operand	6
Math.sqrt(36)	Nonnegative square root	6.0

Additional worked example: Rounding methods are different contracts

ceil moves toward positive infinity; floor moves toward negative infinity. rint chooses the nearest integer with ties to even and returns double. round(double) returns long; for halfway cases it rounds toward positive infinity.

Evaluate rounding at -2.3, 2.5 and 3.5 to expose differences hidden by positive non-tie examples.

Input: Fixed values in the example.

```
System.out.println(Math.ceil(-2.3));
System.out.println(Math.floor(-2.3));
System.out.println(Math.rint(2.5));
System.out.println(Math.rint(3.5));
System.out.println(Math.round(2.5));
```

Call	Result	Interpretation
ceil(-2.3)	-2.0	Toward positive infinity
floor(-2.3)	-3.0	Toward negative infinity
rint(2.5) / round(2.5)	2.0 / 3	Different tie rules

Expected output:

```
-2.0
```

```
-3.0
2.0
4.0
3
```

Independent practice

1. Compute the distance from (0,0) to (6,8) using Math.sqrt. Identify the return type and the arithmetic supplied to it.

2. Predict rint(-2.5) and round(-2.5).

3. Debugging: int length = String.length();

4. Check your understanding: What object is needed for Math.max? What does void mean?

5. Homework: Use Math methods to calculate the hypotenuse for 6 and 8. Read the signatures of every method used.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_14_Built_in_and_instance_methods.tex and PDF/Lecture_14_Built_in_and_instance_methods.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture14.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-12.pptx, slides 7, 8. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 15: User-defined methods

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 6. Learning outcome: CLO-2.

Learning objectives

- Define and call reusable methods

- Choose parameters and return types

- Separate computation from console output

Concepts explained

A method definition

A method header names its return type and parameters. The body defines the computation. A non-void method must return a compatible value on every normal path.

```
static int square(int value) {
  return value * value;
}
```

Calls and returned values

The caller evaluates arguments and passes their values. return ends the current method invocation. The caller can store, print or combine the result.

```
int area = square(5);
System.out.println(square(3) + 1);
```

Void methods

A void method performs an action without returning a value. return; can end a void method early. A caller cannot assign a void result to a variable.

```
static void greet(String name) {
  System.out.println("Hello " + name);
}
```

Method contracts

Choose parameters that represent the required inputs. State preconditions and what the return value means. Keep one clear responsibility per method.

```
Contract: rectangleArea(width,height)
Inputs: nonnegative side lengths
Result: product in square units
```

A result is reusable when calculation stays separate

A calculation method returns a value instead of deciding how it will appear on screen. The caller can print, store or test that result. Keeping console input outside the method also permits deterministic tests.

```
average(60,81) returns 70.5.
The caller decides whether to show one or two decimal places.
```

Correct types matter inside the method

The declared return type does not change integer arithmetic inside an expression. If a and b are int, (a+b)/2 truncates before returning. Convert before arithmetic that must retain a fraction.

```
return ((double) a + b) / 2;
The sum and division use double arithmetic.
```

Key distinctions

Part	Example	Role
Return type	double	Type of result
Method name	average	Operation identifier
Parameters	int a, int b	Local inputs
Return expression	((double)a+b)/2	Computed result

First worked example

```
int result = rectangleArea(5, 3);
System.out.println(result);

static int rectangleArea(int width, int height) {
  return width * height;
}
```

Expected result and interpretation:

```
15
width receives 5 and height receives 3.
return sends their product to the caller.
```

Guided problem and trace

Define static double average(int a, int b). Call it with 60 and 81 and print the result.

1. Define a static method with two integer parameters and a double result.

2. Convert a to double before addition and division.

3. Call the method from main and print the returned result.

```
double result = average(60, 81);
System.out.println(result);

static double average(int a, int b) {
  return ((double) a + b) / 2;
}
```

Execution point	State	Result
Call in main	Arguments 60, 81	Invocation starts
Parameter binding	a=60, b=81	Copies of arguments

Execution point	State	Result
Return calculation	141.0 / 2	70.5
Caller assignment	result receives return	70.5

Expected output:

```
70.5
```

A method is an input-output contract

The caller supplies values; the method returns a result that the caller can reuse.

Relevant labels: Call: average(8,9); Bind a=8, b=9; Compute 17 / 2.0; Return 8.5

Arguments	Correct mean	Reason to test
8, 9	8.5	Fractional answer
0, 0	0.0	Zero case
-4, 4	0.0	Opposite signs

Additional worked example: A reusable sumDigits method

The last decimal digit is n % 10; n / 10 removes it for a nonnegative integer. A method contract must state whether negative inputs are accepted. The loop stops when no digits remain; zero has digit sum zero.

Implement sumDigits(long n) for nonnegative input, and demonstrate the source case 234 -> 9.

Input: Fixed values in the example.

```
System.out.println(sumDigits(234));
System.out.println(sumDigits(0));
```

```
static int sumDigits(long n) {
  if (n < 0) throw new IllegalArgumentException("nonnegative required");
  int sum = 0;
  while (n != 0) {
    sum += n % 10;
    n /= 10;
  }
  return sum;
}
```

n before step	Digit added	sum afterward
234	4	4
23	3	7
2	2	9

Expected output:

```
9
0
```

Independent practice

1. Write a double-returning average method for two ints. Prevent integer addition overflow before dividing.

2. What should happen on -234 under this method contract?

3. Debugging: static int larger(int a, int b) {
if (a > b) return a;
}

4. Check your understanding: How do a parameter and an argument differ? Can a void call appear on the right of an assignment?

5. Homework: Write isEven(int n) returning boolean. Demonstrate calls for -2, 0, 3 and 8.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_15_User_defined_methods.tex and PDF/Lecture_15_User_defined_methods.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture15.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Methods Practice.pptx, slides 2, 3. Added an explicit nonnegative-input contract and rejection path to the source method.

Source exercise deck credits Instructor Khurram Iqbal.

Lecture 16: Parameters, stack and scope

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 6. Learning outcome: CLO-2.

Learning objectives

- Explain pass-by-value and stack frames

- Resolve overloaded calls

- Identify local and block scope

Concepts explained

Pass-by-value

Java copies each argument value into a parameter. Reassigning a primitive parameter does not reassign the caller variable. Reference values also pass by value.

```
static void change(int n) {
  n = 99;
}
// change(x) leaves x unchanged
```

Call stack and activation records

Each active invocation has a stack frame with its own local state. A call suspends the caller until the called method returns normally or throws. A return resumes the caller at the call site.

```
main calls doubleIt(4)
doubleIt has its own parameter n=4
doubleIt returns 8
main stores 8
```

Method overloading

Overloads share a name but use different parameter lists. The compiler selects an applicable overload from argument types. Return type alone cannot distinguish overloads.

```
static int twice(int n) { return n * 2; }
static double twice(double n) { return n * 2; }
```

Variable scope

A local name is available only within its scope. A block variable cannot be used outside its block. A local variable can hide a field with the same name.

```
if (ready) {
  int result = 10;
}
// result is unavailable here
```

Each invocation owns its local variables

A caller's x and a callee's n are distinct variables. Parameter binding copies the argument value into the callee frame. The returned value travels back separately and can be stored in another caller variable.

```
Caller: x=4
Callee: n receives 4, returns 5
Caller after return: x=4, y=5
```

Overloading uses the declared parameter list

Methods with the same name can accept different argument counts or types. The compiler resolves the call using the argument types available at compilation. A different return type alone does not create a new overload.

```
max(int a, int b)
max(int a, int b, int c)
These are distinct signatures.
```

Key distinctions

Property	Scope	Lifetime
Question	Where can a name be used?	How long does state exist?
Local variable	Its enclosing declaration scope	During the invocation
Block variable	Inside the relevant block	During block execution

First worked example

```
int x = 5;
change(x);
System.out.println(x);
System.out.println(twice(3));
System.out.println(twice(3.0));
```

```
static void change(int n) { n = 99; }
static int twice(int n) { return n * 2; }
static double twice(double n) { return n * 2; }
```

Expected result and interpretation:

```
5
6
6.0
change modifies only its parameter. The argument type selects each twice overload.
```

Guided problem and trace

Trace main with x=4 calling addOne(x), storing its returned result in y. Show the caller and callee values before and after return.

1. Keep x in the caller and pass its value to addOne.

2. Return n + 1 from the callee.

3. Store the result in y and print both caller variables.

```
int x = 4;
int y = addOne(x);
System.out.println(x);
System.out.println(y);
```

```
static int addOne(int n) {
  return n + 1;
}
```

Moment	main frame	addOne frame
Before call	x=4, y not assigned	No frame yet
Inside call	x=4, waiting	n=4
At return	Waiting for result	Returns 5
After return	x=4, y=5	Invocation completed

Expected output:

```
4
5
```

A call has its own local state

The parameter is a separate variable. Returning a value does not overwrite the argument.

Relevant labels: main: x = 4; addOne: n = 4; return n + 1 = 5; main: x = 4, y = 5

Moment	Caller x	Callee n
Before call	4	Not active
After n = n + 1	4	5
After return	4	Invocation ended

Additional worked example: Overload resolution with mixed arguments

An overload is selected from the available parameter signatures. An int argument can widen to double when the chosen signature requires it. The desired return type does not resolve which overload to call.

Define int and double max overloads, then call max(3,4) and max(2,2.5).

Input: Fixed values in the example.

```
System.out.println(max(3, 4));
System.out.println(max(2, 2.5));
```

```
static int max(int a, int b) {
  return a >= b ? a : b;
}
static double max(double a, double b) {
  return a >= b ? a : b;
}
```

Call	Selected parameters	Returned type / value
max(3,4)	int, int	int / 4
max(2,2.5)	double, double	double / 2.5

Call	Selected parameters	Returned type / value
max(2.0,2.5)	double, double	double / 2.5

Expected output:

```
4
2.5
```

Independent practice

1. Trace bump(x) when bump increments its parameter and returns it, but main ignores the return value. How must main change to update x?

2. Why can overloads f(int,double) and f(double,int) make f(1,1) ambiguous?

3. Debugging: static int f(int x) { return x; }
static double f(int x) { return x; }

4. Check your understanding: Does changing a parameter change its primitive argument? Is a for-header variable visible after the loop?

5. Homework: Write overloaded max methods for two ints and three ints. Reuse the two-argument version inside the three-argument version.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_16_Parameters_stack_and_scope.tex and PDF/Lecture_16_Parameters_stack_and_scope.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture16.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Lecture-14.pptx, slides 28, 29, 30. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 17: Midterm concept review

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Lectures 1-16. Learning outcome: CLO-1 and CLO-2.

Learning objectives

- Connect concepts from the first half of the course

- Trace expressions, branches and loops

- Explain the cause of a programming error

Concepts explained

Review scope

This review supports the midterm slot in the supplied outline. Topics: source execution, types, control flow, strings and methods. These questions are practice material, not an official examination.

```
Explain each result before running code.
State assumptions about valid inputs.
Use a trace when variables change.
```

Expressions and assignment

An expression result depends on operand types. Assignment stores a value after the expression is evaluated. Parentheses change grouping but do not change operand types.

```
double a = 9 / 2;
double b = 9 / 2.0;
// Explain both values.
```

Control-flow tracing

Record the condition result before entering each branch or loop. Count the final failed while condition check. Trace continue and break as control transfers.

```
int s = 0;
for (int i=1; i<5; i++) {
  if (i==2) continue;
  s += i;
}
```

Strings and methods

String content comparison uses equals. substring uses an exclusive end index. Primitive parameters receive copies of values.

```
"CSC103".substring(0,3)
"CS".equals("cs")
// Explain the type of each result.
```

An integrated answer needs more than syntax

A method signature must match its intended inputs and result. The body must handle ties as well as strict comparisons. A short manually solved case should accompany the implementation.

```
Maximum of 7, 7 and 3 is 7.
A correct solution must preserve ties.
```

Nested calls can be traced in stages

A returned result can become an argument to another call. Tracing the inner result first prevents confusion about execution order. This technique avoids duplicating the comparison logic.

```
max(max(4,9),7)
Inner result: max(4,9) = 9
Outer result: max(9,7) = 9
```

Key distinctions

Practice expression	Result	Reason
9 / 2	4	Both operands are int
9 / 2.0	4.5	One operand is double
"CSC103".substring(0,3)	CSC	End index is excluded
"CS".equals("cs")	false	Case-sensitive content comparison

First worked example

```
int total = 0;
for (int i = 1; i <= 4; i++) {
  total += i * 2;
}
System.out.println(total / 4.0);
```

Expected result and interpretation:

```
5.0
The accumulator values are 2, 6, 12 and 20.
Division by 4.0 produces a double result.
```

Guided problem and trace

In pairs, design a method that returns the larger of two numbers. Use it to find the largest of three values without arrays.

1. Define a two-argument maximum method.

2. Use its returned result as one argument to a second call.

3. Test an ordinary case and a tie case.

```
int result = max(max(4, 9), 7);
System.out.println(result);
System.out.println(max(max(7, 7), 3));
```

```
static int max(int a, int b) {
  return a >= b ? a : b;
}
```

Call	Arguments	Result
First inner call	4 and 9	9
First outer call	9 and 7	9
Tie inner call	7 and 7	7
Tie outer call	7 and 3	7

Expected output:

```
9
7
```

Connect the first-half concepts

Review by tracing how data moves through constructs, not by memorising syntax alone.

Relevant labels: Read a value; Choose a branch; Repeat a step; Return a result

Concept	Question to ask	Typical mistake
Expression	Which arithmetic type?	Integer division
Selection	Which condition matches first?	Incorrect branch order
Method	Where is the return stored?	Ignoring the returned value

Additional worked example: A two-digit lottery decision

Check exact order before reversed order, because the more specific award takes priority. Quotient and remainder separate the tens and units digits. State the two-digit input domain; this worked case fixes the lottery number for tracing.

Use lottery 47 and guess 74. Award 10000 for exact order, 3000 for reversed digits, 1000 for one matching digit, otherwise 0.

Input: Fixed values in the example.

```
int lottery = 47, guess = 74;
int a = lottery / 10, b = lottery % 10;
int c = guess / 10, d = guess % 10;
int award;
if (guess == lottery) award = 10000;
else if (a == d && b == c) award = 3000;
else if (a == c || a == d || b == c || b == d) award = 1000;
else award = 0;
System.out.println(award);
```

Guess (lottery 47)	First matching rule	Award
47	Exact order	10000
74	Reversed order	3000

Guess (lottery 47)	First matching rule	Award
45 / 12	One match / none	1000 / 0

Expected output:

```
3000
```

Independent practice

1. Without arrays, write a method that returns the larger of two values, then use it to find the largest of -3, -8 and -5.

2. Why would checking one-digit matching before exact matching produce a wrong award?

3. Debugging: int n=4;
while(n>0) {
System.out.println(n);
n++;
}

4. Check your understanding: Explain one compile-time error, one runtime exception and one logic error from the first 16 lectures.

5. Homework: Create a one-page revision sheet with one traced example for each topic you find difficult.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_17_Midterm_concept_review.tex and PDF/Lecture_17_Midterm_concept_review.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture17.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Strings -practice.pptx, slides 10. Fixed the lottery at 47; assumes both values are integers in 10..99. This remains a formative midterm exercise.

Source exercise deck credits Instructor Khurram Iqbal.

Lecture 18: Midterm practice workshop

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Lectures 1-16. Learning outcome: CLO-1 and CLO-2.

Learning objectives

- Solve an integrated problem under time constraints

- Justify input and boundary cases

- Review solutions using an explicit rubric

Concepts explained

Practice task and assumptions

Process a fixed number of valid marks in 0..100. Calculate the average and count marks at least 50. Handle zero records without division.

```
Input example: 4 then 40, 50, 60, 70
Expected count passing: 3
Expected average: 55.0
```

Decomposition

Separate reading marks from computing a classification. Choose a counter, sum and passing count. Define what should happen when the count is zero.

```
State: i, sum, passes, count
Repeated step: read one mark, update totals
Final step: report or print No data
```

Boundary test design

A mark of 49 fails the threshold and 50 passes. A count of zero exercises the no-data branch. Equal values make averages easy to verify manually.

```
Case A: count 0
Case B: count 1, mark 50
Case C: count 2, marks 49 and 50
```

Peer-review rubric

Specification and assumptions: 2 practice points. Correct control flow and calculation: 4 points. Boundary tests: 2 points. Clear explanation and style: 2 points.

```
Reviewer: cite one concrete strength.
Reviewer: identify one reproducible failure.
Author: explain the correction.
```

A specification determines the loop state

Reading a fixed number of marks requires a counter. The average requires a sum and a count. A passing count requires a Boolean rule applied to every mark.

```
count = requested number of records
sum = sum of accepted records
passes = number at least 50
```

Zero records need a separate outcome

The loop executes no bodies when the requested count is zero. A zero count cannot support an arithmetic mean. Printing No data states the situation without inventing a numeric result.

```
Input: 0
No mark tokens are read.
Output: No data
```

Key distinctions

Case	Marks	Mean	Passing count
Ordinary	40,50,60,70	55.0	3
Threshold	50	50.0	1
Mixed boundary	49,50	49.5	1
No records	None	No data	0

First worked example

```
int sum = 0, passes = 0;
for (int mark = 40; mark <= 70; mark += 10) {
  sum += mark;
  if (mark >= 50) passes++;
}
System.out.println(sum / 4.0);
System.out.println(passes);
```

Expected result and interpretation:

```
55.0
3
The values are 40, 50, 60 and 70.
This deterministic demonstration substitutes for console input.
```

Guided problem and trace

Write the full console version for a user-specified nonnegative count. Use a method isPass(int mark).

1. Read count and initialise both accumulators.

2. Read count marks, adding to sum and incrementing passes when isPass returns true.

3. Handle count zero separately and otherwise report the mean and passing count.

Input:

```
4 40 50 60 70
```

```
java.util.Scanner in = new java.util.Scanner(System.in);
int count = in.nextInt();
int sum = 0;
```

```
int passes = 0;
for (int i = 0; i < count; i++) {
  int mark = in.nextInt();
  sum += mark;
  if (isPass(mark)) passes++;
}
if (count == 0) System.out.println("No data");
else {
  System.out.println((double) sum / count);
  System.out.println(passes);
}
```

```
static boolean isPass(int mark) {
  return mark >= 50;
}
```

Mark	sum after addition	Pass?	passes
40	40	No	0
50	90	Yes	1
60	150	Yes	2
70	220	Yes	3

Expected output:

```
55.0
3
```

An integrated marks problem has stages

Separate the loop state from the final reporting decision.

Relevant labels: Read count; Read each mark; Update totals; Report summary

Input marks	Mean	Passes (>=50)
49, 50, 51	50.0	2
80	80.0	1
No records	No data	0

Additional worked example: Sorting three values with compare-and-swap

Swap the first pair if out of order, then the second pair, then the first pair again. The third comparison repairs the first pair after the second swap. The method changes local parameter values and prints them; it does not mutate caller primitives.

Adapt the source displaySortedNumbers method and trace inputs 9, 2 and 5.

Input: Fixed values in the example.

```
displaySortedNumbers(9, 2, 5);
```

```
static void displaySortedNumbers(double a, double b, double c) {
  double temp;
```

```
  if (a > b) { temp = a; a = b; b = temp; }
  if (b > c) { temp = b; b = c; c = temp; }
  if (a > b) { temp = a; a = b; b = temp; }
  System.out.println(a + " " + b + " " + c);
}
```

Stage	Values	Reason
Start	9, 2, 5	Unsorted
Compare first pair	2, 9, 5	Swap 9 and 2
Compare second pair	2, 5, 9	Swap 9 and 5
Compare first pair again	2, 5, 9	No swap

Expected output:

```
2.0 5.0 9.0
```

Independent practice

1. Design a three-case test plan for a console marks summary: an ordinary case, the pass boundary, and zero records. Explain the expected results before coding.

2. Which input makes the final comparison necessary?

3. Debugging: double average = sum / count;
// count may be zero; sum and count are int

4. Check your understanding: What test separates >=50 from >50? What part of this solution is reusable without console input?

5. Homework: Revise your workshop solution and attach three tests with expected and actual results.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_18_Midterm_practice_workshop.tex and PDF/Lecture_18_Midterm_practice_workshop.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture18.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Methods Practice.pptx, slides 8, 9. Corrected syntax and supplied fixed inputs for the source sorting exercise; restricted to finite ordinary numbers.

Source exercise deck credits Instructor Khurram Iqbal.

Lecture 19: Recursion and backtracking

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 18. Learning outcome: CLO-2.

Learning objectives

- Identify base and recursive cases

- Trace recursive stack frames

- Explain recursive search and backtracking

Concepts explained

Base case and progress

A recursive method calls itself directly or indirectly. A base case returns without another recursive call. Every permitted input must progress toward a base case.

```
sumTo(0) = 0
sumTo(n) = n + sumTo(n-1), n > 0
```

Recursive call stack

Each recursive call keeps its own parameters and pending work. Returns unwind in reverse order. The deepest base case determines the first returned result.

```
sumTo(3): waits for 3 + sumTo(2)
sumTo(2): waits for 2 + sumTo(1)
sumTo(1): waits for 1 + sumTo(0)
sumTo(0): returns 0
```

Recursion and iteration

Both can express repetition. A loop can often use constant auxiliary space for a simple sum. Recursive depth consumes stack space and can cause StackOverflowError.

```
Iterative sum:
int total = 0;
for (int i=1; i<=n; i++) total += i;
```

Backtracking search

Backtracking explores a choice, then explores alternatives when needed. A base case recognises a complete candidate. Immutable partial results naturally preserve the earlier choice state.

```
binary("", 2)
  binary("0", 1)
    outputs 00, 01
  binary("1", 1)
    outputs 10, 11
```

A recursive definition needs a domain

factorial(0)=1 supplies the base case. For positive n, factorial(n)=n*factorial(n-1). For this int implementation, restrict n to 0..12 so that results fit the chosen type.

```
factorial(4) = 4 * 3 * 2 * 1 = 24
13! exceeds the positive int range.
```

Returning is different from making a call

A factorial frame waits for the smaller factorial before multiplying. The base case provides the first completed answer. Unwinding performs the pending multiplications from the deepest frame upward.

```
fact(0) returns 1
fact(1) returns 1 * 1
fact(2) returns 2 * 1
fact(3) returns 3 * 2
```

Key distinctions

Technique	Stored work	Good fit
Iteration	Loop state and accumulator	Simple repeated accumulation
Direct recursion	Pending work in frames	A problem with a smaller same-form case
Backtracking	Choices and search state	Exploring alternative candidates

First worked example

```
System.out.println(sumTo(4));
binary("", 2);
```

```
static int sumTo(int n) {
  if (n < 0) throw new IllegalArgumentException("negative n");
  if (n == 0) return 0;
  return n + sumTo(n - 1);
}
static void binary(String prefix, int remaining) {
  if (remaining == 0) { System.out.println(prefix); return; }
  binary(prefix + "0", remaining - 1);
  binary(prefix + "1", remaining - 1);
}
```

Expected result and interpretation:

```
10
00
01
10
11
Each binary choice reduces the remaining depth by one.
```

Guided problem and trace

Trace factorial(4) with base case factorial(0)=1. Then write the recursive method for inputs 0..12 using int.

1. Reject n outside the supported range before computing.

2. Return 1 at n==0.

3. For a positive n, return n multiplied by factorial(n-1).

```
System.out.println(factorial(4));
System.out.println(factorial(0));
```

```
static int factorial(int n) {
  if (n < 0 || n > 12) {
    throw new IllegalArgumentException("n must be in 0..12");
  }
  if (n == 0) return 1;
  return n * factorial(n - 1);
}
```

Returning frame	Calculation	Returned value
factorial(0)	Base case	1
factorial(1)	1 * 1	1
factorial(2)	2 * 1	2
factorial(3)	3 * 2	6
factorial(4)	4 * 6	24

Expected output:

```
24
1
```

Recursive calls unwind in reverse order

Calls move toward the base case; returned values then flow back to waiting callers.

Relevant labels: fact(3); fact(2); fact(1); fact(0) = 1

Returning from	Calculation	Returned value
fact(1)	1 * 1	1
fact(2)	2 * 1	2
fact(3)	3 * 2	6

Additional worked example: Fibonacci recursion and repeated work

Two base cases define fib(0)=0 and fib(1)=1. For n>=2, two smaller calls contribute to the result. Naive recursion repeats subproblems; use only small n for the demonstration.

Calculate fib(6), and trace fib(4) to see fib(2) computed more than once.

Input: Fixed values in the example.

```
System.out.println(fib(6));
```

```
static int fib(int n) {
  if (n < 0 || n > 30) throw new IllegalArgumentException("use 0..30");
  if (n < 2) return n;
  return fib(n - 1) + fib(n - 2);
}
```

Index	Recurrence / base	Value
0 / 1	Base cases	0 / 1
2 / 3	1+0 / 1+1	1 / 2
4 / 5 / 6	2+1 / 3+2 / 5+3	3 / 5 / 8

Expected output:

```
8
```

Independent practice

1. Write recursive sumTo(n) for 0 through 100. Trace sumTo(3), including the base-case return.

2. Why does changing the method to tail-recursive form not guarantee constant stack space in Java?

3. Debugging: static int f(int n) { return n + f(n-1); }

4. Check your understanding: Why does backtracking need to restore mutable state? What causes the direct recursive sum to terminate?

5. Homework: Generate all three-character binary strings recursively. Explain why there are eight results.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_19_Recursion_and_backtracking.tex and PDF/Lecture_19_Recursion_and_backtracking.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture19.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Recursion(New).pptx, slides 33, 34, 35; Java Recursion.pptx, slides 6, 7, 12, 13. Added a small demonstration bound. Memoization and tree traversal in the source are optional enrichment, not new assessed prerequisites.

Lecture 20: One-dimensional arrays

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 7. Learning outcome: CLO-2.

Learning objectives

- Create arrays with fixed or runtime sizes

- Traverse elements safely

- Compute an aggregate and explain bounds errors

Concepts explained

Array declaration and creation

An array groups indexed elements of one component type. An array reference and the array object are different things. The array length stays fixed after construction.

```
int[] marks = {60, 75, 90};
int[] counts = new int[4];
```

Indexes and length

Indexes start at zero and end at length-1. length is an array property, without parentheses. An invalid index raises ArrayIndexOutOfBoundsException.

```
marks[0]       // 60
marks.length   // 3
marks[3]       // invalid
```

Array traversal

An indexed loop supports reading and updating elements. An enhanced for loop conveniently reads each value. Keep the condition i < array.length.

```
int total = 0;
for (int value : marks) {
  total += value;
}
```

Runtime size and aggregation

Read or compute a size before new int[size]. Reject negative sizes and unreasonable requests. A mean or maximum needs an explicit empty-array policy.

```
int size = 5;
int[] data = new int[size];
// data has indexes 0..4
```

The initial maximum must come from the domain

Zero is an incorrect default maximum when all values can be negative. For a nonempty array, the first element supplies a valid candidate. Each comparison then maintains the maximum of the processed prefix.

```
Input: {-4,-9,-2}
Start at -4. Compare -9, then -2.
Final maximum: -2
```

A traversal bound is part of correctness

For an array of length n, valid indexes satisfy 0 <= i < n. Starting at 1 is appropriate after using index 0 as the initial candidate. A length-zero input has no first element, so check it before initialisation.

```
int maximum = values[0];
for (int i=1; i<values.length; i++) ...
```

Key distinctions

Index	0	1	2
Element	-4	-9	-2
Valid for length 3?	Yes	Yes	Yes
Candidate after processing	-4	-4	-2

First worked example

```
int[] marks = {60, 75, 90};
int total = 0;
for (int i = 0; i < marks.length; i++) {
  total += marks[i];
}
System.out.println((double) total / marks.length);
```

Expected result and interpretation:

```
75.0
The running sums are 60, 135 and 225.
The loop uses indexes 0, 1 and 2.
```

Guided problem and trace

Find the maximum in {-4,-9,-2}. State the policy for an empty array.

1. Require a nonempty array before accessing values[0].

2. Initialise the maximum from the first element.

3. Visit each remaining element and replace the candidate only when the new value is greater.

```
int[] values = {-4, -9, -2};
System.out.println(maximum(values));

static int maximum(int[] values) {
  if (values == null || values.length == 0) {
    throw new IllegalArgumentException("nonempty array required");
  }
  int maximum = values[0];
  for (int i = 1; i < values.length; i++) {
    if (values[i] > maximum) maximum = values[i];
```

```
  }
  return maximum;
}
```

Step	Compared value	Previous maximum	New maximum
Initialisation	values[0] = -4	None	-4
i = 1	-9	-4	-4
i = 2	-2	-4	-2

Expected output:

```
-2
```

An array stores indexed elements

A length-4 array has indexes 0 through 3. Index 4 is outside the array.

Relevant labels: 12; 7; 19; 4

Index	Element	Running sum after visit
0	12	12
1	7	19
2, then 3	19, then 4	38, then 42

Additional worked example: Linear search and the first match

Compare the target with successive elements until the first match. Return its index, not the element value. Use -1 when no match exists. Linear search works without sorting; binary search has a sorted-input precondition.

Search {5,8,12,3,9} for 12 and for 7, using the source search contract.

Input: Fixed values in the example.

```
int[] values = {5, 8, 12, 3, 9};
System.out.println(linearSearch(values, 12));
System.out.println(linearSearch(values, 7));
```

```
static int linearSearch(int[] values, int key) {
  for (int i = 0; i < values.length; i++) {
    if (values[i] == key) return i;
  }
  return -1;
}
```

Index visited	Value	Searching for 12
0	5	Continue
1	8	Continue

Index visited	Value	Searching for 12
2	12	Return 2

Expected output:

```
2
-1
```

Independent practice

1. Write a loop to count the values above 10 in {12,7,19,4}. Give the count and identify the indexes that qualify.

2. Predict searching {12,5,12} for 12 and searching an empty array.

3. Debugging: for(int i=0; i<=a.length; i++) {
System.out.println(a[i]);
}

4. Check your understanding: What values does new int[3] contain? Does changing the enhanced-for variable update the array?

5. Homework: Read five temperatures into an array. Print the mean and number above the mean.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_20_One_dimensional_arrays.tex and PDF/Lecture_20_One_dimensional_arrays.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture20.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Arrays1D(new).pptx, slides 30, 31, 32; Logic Building, Dry Run Techniques, and Flagged While Loops.pptx, slides 13. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 21: Arrays and method arguments

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 7. Learning outcome: CLO-2.

Learning objectives

- Pass and return arrays

- Distinguish mutation from parameter reassignment

- Use varargs and command-line arguments

Concepts explained

Passing an array reference

A parameter receives a copy of the array reference. Both references can reach the same array object. Changing an element is visible to the caller.

```
static void setFirst(int[] a) {
  a[0] = 99;
}
```

Reassignment and returned arrays

Reassigning the parameter does not reassign the caller reference. A method can construct and return a new array. A copy can protect the original data from later mutations.

```
static int[] pair(int a, int b) {
  return new int[]{a, b};
}
```

Variable-length argument lists

int... values accepts zero or more int arguments. Inside the method, values behaves as an array. A varargs parameter must be last.

```
static int sum(int... values) {
  int total = 0;
  for (int v : values) total += v;
  return total;
}
```

Command-line arguments

main receives command-line tokens as String elements. Check args.length before indexing. Parse a numeric argument before arithmetic.

```
java Program 10 20

args[0] is "10"
args[1] is "20"
```

A new output array preserves the input

An output array can hold transformed values while the caller keeps the original. The method allocates storage of the same length and writes each result by index. The returned reference identifies the new array.

```
Input array: {2,3}
Output array: {4,6}
The two arrays have different identities.
```

Reference copying does not copy elements

Assigning int[] b=a makes b reach the same array. Creating new int[a.length] makes separate element storage. The choice determines whether later element updates are shared.

```
int[] alias = a;       // same array
int[] copy = a.clone(); // independent primitive elements
```

Key distinctions

Operation	New array?	Caller sees element changes?
b = a	No	Yes, shared array
a[0] = 9	No	Yes, element mutation
parameter = new int[2]	Yes	Caller variable unchanged
return new int[2]	Yes	Caller can store returned reference

First worked example

```
int[] data = {1, 2};
modify(data);
System.out.println(data[0]);
System.out.println(sum(2, 3, 4));

static void modify(int[] a) {
  a[0] = 9;
  a = new int[]{7};
}
static int sum(int... values) {
  int total = 0;
  for (int v : values) total += v;
  return total;
}
```

Expected result and interpretation:

```
9
9
modify changes the shared first element.
Its later parameter reassignment leaves the caller reference unchanged.
```

Guided problem and trace

Write a method returning a new doubled array while leaving the supplied array unchanged. Test {2,3}.

1. Allocate an output array with input.length elements.

2. Calculate each doubled value into the corresponding output position.

3. Return the output and show that the original remains unchanged.

```
int[] original = {2, 3};
int[] result = doubled(original);
System.out.println(java.util.Arrays.toString(result));
System.out.println(java.util.Arrays.toString(original));
```

```
static int[] doubled(int[] input) {
  int[] output = new int[input.length];
  for (int i = 0; i < input.length; i++) {
    output[i] = input[i] * 2;
  }
  return output;
}
```

Index	Input value	Output calculation	Output value
0	2	2 * 2	4
1	3	3 * 2	6
After return	Input still {2,3}	New array reference	{4,6}

Expected output:

```
[4, 6]
[2, 3]
```

A parameter copies an array reference

Both references reach one array, so an element update is visible to the caller.

Relevant labels: caller: data; parameter: a; One array: [2, 3]; a[0] = 9 changes shared storage

Inside the method	Caller observes	Reason
a[0] = 9	Element becomes 9	Shared array
a = new int[]{7}	Original array unchanged	Only parameter reassigned
return a	Caller may store it	Explicit returned reference

Additional worked example: Copying a slice into another array

Array assignment copies a reference; arraycopy copies elements between arrays. Specify source position, target position and the number of elements. For primitive elements the copied values are independent; reference elements would still be references.

Copy elements 20 and 30 from {10,20,30,40} into positions 0 and 1 of a new length-3 int array.

Input: Fixed values in the example.

```
int[] source = {10, 20, 30, 40};
int[] target = new int[3];
System.arraycopy(source, 1, target, 0, 2);
source[1] = 99;
System.out.println(java.util.Arrays.toString(target));
```

Copy step	Source element	Target position
First	source[1] = 20	target[0]
Second	source[2] = 30	target[1]
Not copied	Default zero	target[2]

Expected output:

```
[20, 30, 0]
```

Independent practice

1. Write a method that changes the first element to 9, then reassigns its parameter to a new array. Predict the caller array {2,3} afterward.

2. Why does the later source[1]=99 not change target[0]?

3. Debugging: static void replace(int[] a) { a = new int[]{7}; }
// The caller expects its variable to become {7}.

4. Check your understanding: Does passing an array copy all its elements? Where must a varargs parameter occur?

5. Homework: Create a program that sums integer command-line arguments. Handle zero arguments explicitly.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_21_Arrays_and_method_arguments.tex and PDF/Lecture_21_Arrays_and_method_arguments.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture21.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Arrays1D(new).pptx, slides 21, 22, 23, 24, 25, 27. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 22: Two-dimensional arrays

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 7. Learning outcome: CLO-2.

Learning objectives

- Create and access a table of values

- Traverse rows and columns

- Compute row and column totals

Concepts explained

Arrays of arrays

A Java two-dimensional array is an array of row references. data[r][c] selects column c in row r. A rectangular example has equal row lengths.

```
int[][] marks = {
  {60, 70, 80},
  {50, 65, 75}
};
```

Nested traversal

Use the outer length for row count. Use each row length for its column count. Keep the row and column indexes distinct.

```
for (int r=0; r<data.length; r++) {
  for (int c=0; c<data[r].length; c++) {
    System.out.println(data[r][c]);
  }
}
```

Row totals

Reset a row accumulator at the start of each row. The inner loop adds elements from that row. A row mean divides by the row length only when nonempty.

```
for (int[] row : data) {
  int total = 0;
  for (int value : row) total += value;
  System.out.println(total);
}
```

Column totals

Column traversal reverses the roles of the indexes. A rectangular-matrix contract simplifies column totals. Check shape assumptions before using data[0].

```
For {{1,2},{3,4}}:
Column 0 total: 1 + 3 = 4
Column 1 total: 2 + 4 = 6
```

Different traversals answer different questions

A row traversal keeps the row fixed while the column changes. A column traversal keeps the column fixed while the row changes. A main-diagonal traversal selects entries whose row and column indexes match.

```
For {{2,4},{6,8}}:
Row 0: 2,4
Column 0: 2,6
Main diagonal: 2,8
```

Accumulator placement changes the meaning

A grand total starts before both loops. A row total resets inside the outer loop. A diagonal sum needs one selected element per suitable row.

```
Grand total: 2 + 4 + 6 + 8 = 20
Diagonal total: 2 + 8 = 10
```

Key distinctions

Row / column	Column 0	Column 1	Row total
Row 0	2	4	6
Row 1	6	8	14
Column total	8	12	20 overall

First worked example

```
int[][] data = {{1, 2, 3}, {4, 5, 6}};
for (int[] row : data) {
  int total = 0;
  for (int value : row) total += value;
  System.out.println(total);
}
```

Expected result and interpretation:

```
6
15
A fresh total starts at zero for each row.
```

Guided problem and trace

Compute the total of every element in {{2,4},{6,8}}. Then compute its main diagonal sum.

1. Traverse every element with nested loops to accumulate the grand total.

2. For this square matrix, add data[i][i] in a separate loop.

3. Report each result with a distinct label.

```
int[][] data = {{2, 4}, {6, 8}};
int total = 0;
for (int[] row : data) {
  for (int value : row) total += value;
}
int diagonal = 0;
for (int i = 0; i < data.length; i++) {
```

```
  diagonal += data[i][i];
}
System.out.println("Total: " + total);
System.out.println("Diagonal: " + diagonal);
```

Selected element	Value	Running grand total
data[0][0]	2	2
data[0][1]	4	6
data[1][0]	6	12
data[1][1]	8	20

Expected output:

```
Total: 20
Diagonal: 10
```

Rows and columns identify one element

For this 2 by 3 array, data[1][2] is 5: row 1, column 2.

Relevant labels: 2; 4; 6; 1; 3; 5

Aggregate	Elements	Result
Row 0	2 + 4 + 6	12
Column 1	4 + 3	7
All elements	2 + 4 + 6 + 1 + 3 + 5	21

Additional worked example: Grading a multiple-choice test

Rows represent students and columns represent questions. Compare each answer with the key at the same column index. Reset the score for each student, so scores do not accumulate across rows.

Use the first three source students and answer key {D,B,D,C,C}. Compute each score out of five.

Input: Fixed values in the example.

```
char[][] answers = {
  {'A','B','A','C','C'},
  {'D','B','A','B','C'},
  {'E','D','D','A','C'}
};
char[] key = {'D','B','D','C','C'};
for (int student = 0; student < answers.length; student++) {
  int score = 0;
  for (int question = 0; question < key.length; question++) {
    if (answers[student][question] == key[question]) score++;
  }
  System.out.println("Student " + (student + 1) + ": " + score);
}
```

Student	Correct question numbers	Score
1	2, 4, 5	3
2	1, 2, 5	3
3	3, 5	2

Expected output:

```
Student 1: 3
Student 2: 3
Student 3: 2
```

Independent practice

1. For {{2,4,6},{1,3,5}}, compute column 1 using a loop over rows. State the shape assumption.

2. Which assumption must be checked when rows come from an input file?

3. Debugging: for(int c=0; c<data.length; c++) {
total += data[r][c];
}

4. Check your understanding: What does data.length measure? Where should a row-total accumulator be reset?

5. Homework: Represent marks for three students in two quizzes. Print each student total and each quiz average.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_22_Two_dimensional_arrays.tex and PDF/Lecture_22_Two_dimensional_arrays.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture22.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Multi-dimensional arrays.pptx, slides 71, 72. Uses the first three of the ten supplied students; retains the original five-question key and answers. Replaced typographic quotes with Java quotes.

Lecture 23: Ragged and multidimensional arrays

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 7. Learning outcome: CLO-2.

Learning objectives

- Pass a two-dimensional array to a method

- Handle ragged rows

- Explain copying and additional dimensions

Concepts explained

Two-dimensional parameters

A method can accept int[][] in its parameter list. As with other arrays, the reference value is copied. Element changes can affect caller-visible data.

```
static int total(int[][] data) {
  int sum = 0;
  for (int[] row : data)
    for (int v : row) sum += v;
  return sum;
}
```

Ragged arrays

Rows may have different lengths. Allocate the outer array and each row separately when needed. A newly created outer reference array starts with null rows.

```
int[][] groups = new int[2][];
groups[0] = new int[]{1};
groups[1] = new int[]{2, 3, 4};
```

Shallow and deeper copying

Copying only the outer array preserves shared row references. Changing a shared row element affects both structures. Independent rows require copying each row too.

```
int[][] copy = original.clone();
copy[0][0] = 99;
// original[0][0] is now 99
```

Additional dimensions

Each extra dimension introduces another array level. Give every index a domain meaning. Prefer the smallest number of dimensions that expresses the task.

```
int[][][] attendance = new int[2][3][4];
// [section][week][session]
```

A ragged array needs a row-specific bound

The outer length counts row references, not every stored element. Each non-null row supplies its own length. Adding the row lengths counts all primitive elements, including zero for an empty row.

```
{{1},{2,3},{}} has row lengths 1,2,0.
Element count = 1 + 2 + 0 = 3.
```

Copy depth follows the structure

Cloning a two-dimensional array copies the outer row references. Cloning each row as well produces separate primitive element storage. A deeper structure may require further copying decisions.

```
int[][] copy = new int[a.length][];
for (int r=0; r<a.length; r++)
  copy[r] = a[r].clone();
```

Key distinctions

Row	Contents	Length	Valid indexes
0	{1}	1	0
1	{2,3}	2	0,1
2	{}	0	None

First worked example

```
int[][] data = {{1, 2}, {3}};
System.out.println(total(data));
int[][] copy = data.clone();
copy[0][0] = 9;
System.out.println(data[0][0]);

static int total(int[][] data) {
  int sum = 0;
  for (int[] row : data)
    for (int v : row) sum += v;
  return sum;
}
```

Expected result and interpretation:

```
6
9
The total method processes unequal row lengths.
clone copied the outer references, so the first row remains shared.
```

Guided problem and trace

Write a method to count all elements in a non-null ragged array with non-null rows. Test {{1},{2,3},{}}.

1. Assume the outer array and every row are non-null.

2. Visit each row and add its length to a counter.

3. Return the accumulated count.

```
int[][] values = {{1}, {2, 3}, {}};
System.out.println(elementCount(values));

static int elementCount(int[][] values) {
  int count = 0;
  for (int[] row : values) {
```

```
    count += row.length;
  }
  return count;
}
```

Row visited	Length added	Count before	Count after
0	1	0	1
1	2	1	3
2	0	3	3

Expected output:

```
3
```

Ragged rows have different lengths

The outer length counts rows; each row supplies its own element count.

Relevant labels: 8; 2; 4; 6; Empty row

Row	Values	Length
0	8	1
1	2, 4, 6	3
2	No elements	0

Additional worked example: Selecting the row with the largest sum

Compute one row sum at a time using that row's length. Initialise the best sum from an actual row rather than zero if negatives are allowed. A strict > update keeps the first row when sums tie.

Find the row with the largest sum in the source matrix {{10,20,30},{40,50,60},{70,80,90}}.

Input: Fixed values in the example.

```
int[][] data = {{10,20,30}, {40,50,60}, {70,80,90}};
int bestRow = 0;
long bestSum = Long.MIN_VALUE;
for (int row = 0; row < data.length; row++) {
  long sum = 0;
  for (int value : data[row]) sum += value;
  if (sum > bestSum) { bestSum = sum; bestRow = row; }
}
System.out.println("Row " + bestRow + ", sum " + bestSum);
```

Row index	Row sum	Best row afterward
0	60	0
1	150	1
2	240	2

Expected output:

```
Row 2, sum 240
```

Independent practice

1. For {{8},{2,4,6},{}}, compute the total number of elements and their sum. Assume all rows are non-null.

2. How would you extend the contract to an empty outer array and null rows?

3. Debugging: int[][] a = new int[3][];
a[0][0] = 5;

4. Check your understanding: Does outer-array clone duplicate each row? How many values are in new int[2][3][4]?

5. Homework: Write a method that makes an independent copy of a ragged int[][] with non-null rows. Demonstrate that mutation of the copy leaves the original unchanged.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_23_Ragged_and_multidimensional_arrays.tex and PDF/Lecture_23_Ragged_and_multidimensional_arrays.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture23.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Multi-dimensional arrays.pptx, slides 10, 66, 69. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 24: Exception handling

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 11. Learning outcome: CLO-3.

Learning objectives

- Explain propagation and the exception hierarchy

- Use try, throw and catch

- Distinguish checked and unchecked exceptions

Concepts explained

Exceptional control flow

An exception interrupts the normal flow at the failure point. Java searches for an applicable handler. Statements after the failure in the same try body are skipped.

```
try {
  int n = Integer.parseInt("cat");
  System.out.println(n);
} catch (NumberFormatException ex) {
  System.out.println("Invalid integer");
}
```

Exception hierarchy

Throwable is the common base for Error and Exception. RuntimeException is a subclass of Exception. Most application recovery targets specific Exception subclasses.

```
Throwable
  Error
  Exception
    RuntimeException
```

Checked and unchecked exceptions

Checked exceptions must be caught or declared with throws. Unchecked exceptions do not have that compiler requirement. The distinction is about checking, not whether an event occurs at runtime.

```
Checked: IOException
Unchecked: NumberFormatException
Unchecked: IllegalArgumentException
```

Throwing and catching

throw raises a particular exception object. throws appears in a method declaration. Place specific catches before broader compatible catches.

```
if (marks < 0 || marks > 100) {
  throw new IllegalArgumentException(
      "marks outside 0..100");
}
```

Validation and parsing solve different problems

Parsing checks whether text can represent a number of the requested kind. Validation checks whether the parsed number belongs to the application's allowed domain. The value 101 parses as int but violates a marks range of 0..100.

```
"cat": NumberFormatException while parsing
"101": parses, then fails range validation
"100": passes both checks
```

Exception handling changes the path

throw immediately interrupts the ordinary path in the current block. A compatible catch can report the failure and allow later work to continue. A loop with a try-catch inside its body can examine later inputs after one failure.

```
For inputs -1,0,100,101:
Reject the first, accept the next two, reject the last.
```

Key distinctions

Input	Parses as int?	Valid mark?	Outcome
"cat"	No	Not evaluated	Parsing failure
"-1"	Yes	No	Range failure
"0"	Yes	Yes	Accept
"100"	Yes	Yes	Accept
"101"	Yes	No	Range failure

First worked example

```
try {
  int marks = Integer.parseInt("bad");
  System.out.println(marks);
} catch (NumberFormatException ex) {
  System.out.println("Invalid integer");
}
System.out.println("Finished");
```

Expected result and interpretation:

```
Invalid integer
Finished
The handler runs, then normal execution continues after the try-catch.
```

Guided problem and trace

Define validateMark(int marks) that throws IllegalArgumentException outside 0..100. Test -1, 0, 100 and 101.

1. Write a validator that throws when marks are outside 0..100.

2. Call the validator for each boundary example inside a try block.

3. Print a success message only after validation returns normally.

```
int[] cases = {-1, 0, 100, 101};
for (int mark : cases) {
  try {
    validateMark(mark);
    System.out.println(mark + " accepted");
  } catch (IllegalArgumentException ex) {
    System.out.println(mark + " rejected");
  }
}
```

```
static void validateMark(int mark) {
  if (mark < 0 || mark > 100) {
    throw new IllegalArgumentException("marks outside 0..100");
  }
}
```

Mark	Invalid condition	Control path	Printed result
-1	true	throw then catch	-1 rejected
0	false	Normal return	0 accepted
100	false	Normal return	100 accepted
101	true	throw then catch	101 rejected

Expected output:

```
-1 rejected
0 accepted
100 accepted
101 rejected
```

An exception transfers control to a handler

When parsing fails, the remaining statements in that try block are skipped.

Relevant labels: parse succeeds?; Use integer; Matching catch; Continue after try/catch

Token	Parsing outcome	Handler needed?
42	int 42	No
four	NumberFormatException	Yes
2147483648	Outside int range	Yes

Additional worked example: Which exception is observed first?

One statement can contain more than one potential failure. Execution order determines the observed exception. For a simple array assignment, the right-hand expression is evaluated before the final bounds check. Only the first thrown exception reaches a matching handler in this execution.

Trace int[] a=new int[5]; a[5]=30/zero with zero=0, then compare the case zero=2.

Input: Fixed values in the example.

```
int[] a = new int[5];
int zero = 0;
```

```
try {
  a[5] = 30 / zero;
} catch (ArithmeticException ex) {
  System.out.println("Arithmetic exception");
} catch (ArrayIndexOutOfBoundsException ex) {
  System.out.println("Array index exception");
}
System.out.println("Rest of the code");
```

Divisor / index	First failure	Handler
0 / 5	Integer division by zero	ArithmeticException
2 / 5	Invalid index after RHS	ArrayIndexOutOfBoundsException
2 / 4	None	Assignment succeeds

Expected output:

```
Arithmetic exception
Rest of the code
```

Independent practice

1. Parse "four" inside try/catch, print "Invalid integer" on failure, then print "Finished" after the handler. Predict both output lines.

2. What happens if catch(Exception ex) precedes both specific handlers?

3. Debugging: try { work(); }
catch (Exception ex) { }
catch (NumberFormatException ex) { }

4. Check your understanding: Does throws itself create an exception? Must unchecked exceptions be declared?

5. Homework: Read one integer and report malformed input using a specific catch. Keep parsing and range validation distinguishable.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_24_Exception_handling.tex and PDF/Lecture_24_Exception_handling.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture24.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Exception handling(updated).pptx, slides 29, 30. Made the divisor a variable and verified simple-assignment evaluation order against Java 17 JLS 15.26.1.

Lecture 25: Cleanup and exception propagation

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Deitel Ch 11. Learning outcome: CLO-3.

Learning objectives

- Trace nested handlers and finally

- Rethrow or translate an exception

- Preserve the cause of a failure

Concepts explained

Propagation across calls

An unhandled exception leaves the current method. The runtime continues searching in active callers. Stack traces report the chain of calls involved in the failure.

```
main calls loadCount
loadCount calls parseInt
parseInt throws NumberFormatException
A matching caller catch handles it
```

The finally clause

finally normally runs when control leaves try-catch. It can run after a return or an exception. It is not guaranteed after forced process termination or a JVM crash.

```
try {
  System.out.println("work");
} finally {
  System.out.println("cleanup");
}
```

Rethrowing an exception

A handler may record useful context and rethrow the same exception. Rethrowing lets a higher layer decide how to recover. Avoid duplicate reports at every layer.

```
catch (NumberFormatException ex) {
  throw ex;
}
```

Chained exceptions

Translation can express failure in the caller domain. Pass the original exception as the cause. Preserved causes help explain the underlying problem.

```
catch (NumberFormatException ex) {
  throw new IllegalArgumentException(
      "Invalid record count", ex);
}
```

A handler can pass responsibility upward

A catch may perform a local action and then rethrow the same exception. Its finally block still runs while control leaves the protected structure. An outer handler can then decide how the whole operation should respond.

```
Inner work prints A.
Inner catch prints B and rethrows.
Inner finally prints C.
Outer catch prints D.
```

A cause preserves diagnostic history

A translated exception can state which application operation failed. Its cause identifies the lower-level failure that explains why. Replacing an exception with a message alone discards that useful chain.

```
throw new IllegalArgumentException(
    "Invalid mark field", originalException);
```

Key distinctions

Action in catch	Original failure retained?	Where control goes next
Handle and finish	Available in local catch	After try-catch-finally
throw ex	Same exception	Outer compatible handler
Throw wrapper with cause	As a cause	Outer compatible handler
Return from finally	Can suppress failure	Caller receives the return

First worked example

```
try {
  readCount("bad");
} catch (IllegalArgumentException ex) {
  System.out.println(ex.getMessage());
  System.out.println(ex.getCause().getClass().getSimpleName());
} finally {
  System.out.println("Cleanup");
}
```

```
static int readCount(String text) {
  try { return Integer.parseInt(text); }
  catch (NumberFormatException ex) {
    throw new IllegalArgumentException("Invalid record count", ex);
  }
}
```

Expected result and interpretation:

```
Invalid record count
NumberFormatException
Cleanup
The wrapper retains the original parse failure as its cause.
```

Guided problem and trace

Trace a try body that prints A and throws, a matching catch that prints B, and a finally block that prints C. What if the catch rethrows?

1. Place the failing operation inside an inner try.

2. Print B in its catch, rethrow, and print C in its finally.

3. Use an outer catch to show that propagation continues after cleanup.

```
try {
  try {
    System.out.println("A");
    Integer.parseInt("bad");
  } catch (NumberFormatException ex) {
    System.out.println("B");
    throw ex;
  } finally {
    System.out.println("C");
  }
} catch (NumberFormatException ex) {
  System.out.println("D");
}
```

Order	Location	Action	Output
1	Inner try	Start then parse failure	A
2	Inner catch	Report then rethrow	B
3	Inner finally	Cleanup path	C
4	Outer catch	Handle propagated failure	D

Expected output:

```
A
B
C
D
```

Preserve the cause when adding context

A higher-level message explains the failed operation while the cause retains the original failure.

Relevant labels: parseInt fails; Catch original; Wrap with context; Caller inspects cause

Exception part	Example	Purpose
Wrapper type	IllegalArgumentException	Caller-facing contract
Wrapper message	Invalid record count	Operation context
Cause	NumberFormatException	Underlying failure

Additional worked example: Rethrowing preserves the failure

A handler may observe a failure and rethrow the same exception object. The caller then chooses whether to handle the propagated failure. A finally block runs during normal exception propagation; it must not suppress the pending exception.

Adapt the source test1/test2 example: fail in one method, print context in another, then catch in main.

Input: Fixed values in the example.

```
try {
  relay();
} catch (ArithmeticException ex) {
  System.out.println("Caught in main: " + ex.getMessage());
}
```

```
static void fail() {
  System.out.println("Failure begins");
  throw new ArithmeticException("demo failure");
}
static void relay() {
  try { fail(); }
  catch (ArithmeticException ex) {
    System.out.println("Relay observed failure");
    throw ex;
  } finally { System.out.println("Relay cleanup"); }
}
```

Execution point	Action	Next destination
fail()	Throw ArithmeticException	relay catch
relay catch	Print then rethrow	finally
finally	Print cleanup	main catch

Expected output:

```
Failure begins
Relay observed failure
Relay cleanup
Caught in main: demo failure
```

Independent practice

1. Wrap a parsing failure as IllegalArgumentException with message "Invalid count". Preserve the original exception as its cause.

2. How does throw ex differ from wrapping ex in a new exception?

3. Debugging: finally { return 0; }
// The try body may throw an important exception.

4. Check your understanding: What information does a cause preserve? Does finally guarantee recovery?

5. Homework: Wrap a numeric parsing failure with the input-field name and original cause. Show both messages.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_25_Cleanup_and_exception_propagation.tex and PDF/Lecture_25_Cleanup_and_exception_propagation.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture25.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Exception handling(updated).pptx, slides 35, 37, 38, 39. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 26: Programming environments and debugging

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Reference material. Learning outcome: CLO-3.

Learning objectives

- Use breakpoints and variable inspection

- Distinguish debugging from refactoring

- Refactor while preserving observable behaviour

Concepts explained

A modern programming environment

An IDE integrates editing, compilation, navigation and debugging. Project settings select a JDK and source level. A clean build reduces confusion from stale compiled files.

```
Before a demo:
Check the active JDK.
Check the working directory.
Run the intended main class.
```

Debugger operations

A breakpoint pauses before a selected statement executes. Step over executes a call without entering it. Step into enters a call. Inspect variables and stack frames to test a hypothesis.

```
Hypothesis: the loop misses its final element.
Breakpoint: loop condition
Watch: i, values.length, total
```

Refactoring

Refactoring improves internal structure while preserving intended observable behaviour. Examples include renaming and extracting a method. Run existing tests before and after the change.

```
Before: repeated discount formula
After: one discount(price, rate) method
Same inputs should produce the same outputs.
```

A repeatable debugging process

Reproduce the failure with the smallest useful input. Predict the faulty state, inspect it and make a focused correction. Keep a regression case that demonstrates the original bug.

```
Failure: max({-4,-2}) returns 0.
Cause: max starts at 0.
Correction: initialise from the first element.
```

A minimal failing case guides a repair

The smallest example that violates the contract is easier to reason about. For a maximum function initialised to zero, {-2} already proves the defect. The repair needs an empty-input policy before using the first element.

```
Failure: max({-2}) returns 0.
Required result: -2.
A positive-only test hides this defect.
```

Extraction should preserve the calculation

Choose the statements that implement one responsibility. Replace their external inputs with parameters and their result with a return value. Run the same examples before and after extraction to check behaviour.

```
Before: loop embedded in main
After: main calls max(values)
The same array should yield the same maximum.
```

Key distinctions

Observation	Hypothesis	Debugger check
Negative-only case gives 0	Bad initial candidate	Inspect maximum before loop
Last value ignored	Wrong upper bound	Watch final index
Total changes between calls	Shared state retained	Inspect field or accumulator

First worked example

```
int[] values = {-4, -2, -9};
int maximum = values[0];
for (int i = 1; i < values.length; i++) {
  if (values[i] > maximum) maximum = values[i];
}
System.out.println(maximum);
```

Expected result and interpretation:

```
-2
Initial maximum: -4. After comparing -2: -2.
Comparing -9 leaves the maximum unchanged.
```

Guided problem and trace

Extract the demonstrated maximum calculation into max(int[] values). Specify empty-input behaviour and test negative-only data.

1. Document that the method requires a nonempty array.

2. Move the maximum calculation into a method returning int.

3. Call it for negative-only data and reject the empty case explicitly.

```
System.out.println(max(new int[]{-4, -2, -9}));
try {
  max(new int[]{});
} catch (IllegalArgumentException ex) {
  System.out.println("Empty array rejected");
}
```

```
static int max(int[] values) {
  if (values == null || values.length == 0) {
    throw new IllegalArgumentException("nonempty array required");
  }
  int maximum = values[0];
  for (int i = 1; i < values.length; i++) {
    if (values[i] > maximum) maximum = values[i];
  }
  return maximum;
}
```

Breakpoint	Observed value	Interpretation
Before loop	maximum = -4	Valid first candidate
After i=1	maximum = -2	Larger value replaces it
After i=2	maximum = -2	-9 cannot replace it
Empty input guard	length = 0	Exception before index access

Expected output:

```
-2
Empty array rejected
```

Debugging is a hypothesis-checking cycle

Inspect the first state that differs from your prediction, then rerun the reproducing case.

Relevant labels: Reproduce; Pause and inspect; Explain cause; Fix and retest

Observation	Hypothesis	Useful inspection
Last element omitted	Loop stops too early	Index and length at condition
First item skipped	Counter starts at 1	Initial counter value
Total persists	Accumulator not reset	Value before the next run

Additional worked example: Debugging an incorrect array average

The source debugger exercise starts its sum loop at 1 and omits the first element. It also divides ints before returning double, so a fractional mean can be lost. Use a fractional expected result to detect both defects instead of only testing an integer-valued mean.

Correct the sum and average for {1,2,3,4}. Inspect i, sum and the final divisor with a breakpoint in the loop.

Input: Fixed values in the example.

```
int[] values = {1, 2, 3, 4};
if (values.length == 0) throw new IllegalArgumentException("empty");
int sum = 0;
for (int i = 0; i < values.length; i++) sum += values[i];
double average = (double) sum / values.length;
System.out.println(sum);
System.out.println(average);
```

Implementation	Sum	Returned average
Starts at 1; integer division	9	2.0
Index fixed only	10	2.0
Index and division fixed	10	2.5

Expected output:

```
10
2.5
```

Independent practice

1. A sum loop uses i < values.length - 1 for {2,4,6}. Predict the wrong result, identify the missing visit, and correct the condition.

2. Why might {1,2,3,4,5} hide the integer-division defect after the loop is fixed?

3. Debugging: int maximum = 0;
for (int v : values) maximum = Math.max(maximum, v);

4. Check your understanding: Does rename refactoring intentionally change output? What is a regression test?

5. Homework: Record one debugging session: failing input, hypothesis, observed variable values, correction and regression test.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides:
ISB_Enhanced_Beamer/Lecture_26_Programming_environments_and_debugging.tex and PDF/Lecture_26_Programming_environments_and_debugging.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture26.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Introduction&VScode setup.pptx, slides 13. Changed the source five-element fixture to four elements so the fractional-average defect remains observable; added the empty-input check.

Lecture 27: Library components and APIs

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Reference material. Learning outcome: CLO-3.

Learning objectives

- Read an API signature and contract

- Use library methods without guessing side effects

- Choose and verify a suitable component

Concepts explained

API contracts

An API exposes operations that another program may call. Read parameter types, return type and documented failures. Check preconditions and side effects.

```
Arrays.sort(int[] a)
Result type: void
Effect: sorts the supplied array
Caller-visible mutation: yes
```

Imports and qualification

An import lets a source file use a shorter type name. It does not copy library code into the source. java.lang types such as Math are available without an explicit import.

```
import java.util.Arrays;

Arrays.sort(values);
```

Array utilities

Arrays.sort rearranges elements into ascending order. Arrays.copyOf can create an independent primitive array. Arrays.toString provides readable element output.

```
int[] copy = java.util.Arrays.copyOf(a, a.length);
java.util.Arrays.sort(copy);
```

Searching with a precondition

Arrays.binarySearch requires appropriately sorted input. A nonnegative result is a matching index. A negative result encodes an insertion point and means absent.

```
Sorted input: {2,5,8}
Search 5: index 1
Search 7: negative result
```

Preserving order is a design requirement

Sorting in place changes the supplied array. If the original sequence has meaning, copy the primitive elements before sorting. The sorted copy can then satisfy a search precondition without changing the original.

```
Original sequence: {9,1,4}
Sorted copy: {1,4,9}
Original sequence remains {9,1,4}.
```

A search result must be interpreted by its contract

binarySearch returns a matching index when the key exists. When absent, it returns -(insertion point)-1. Never use a negative result directly as an array index.

```
In {1,4,9}, key 6 has insertion point 2.
Returned value: -2 - 1 = -3.
```

Key distinctions

Library call	Modifies input?	What it returns
Arrays.sort(a)	Yes	void
Arrays.copyOf(a,n)	No	A new array
Arrays.toString(a)	No	Readable String
Arrays.binarySearch(a,key)	No	Match index or encoded absence

First worked example

```
int[] data = {8, 2, 5};
int[] copy = java.util.Arrays.copyOf(data, data.length);
java.util.Arrays.sort(copy);
System.out.println(java.util.Arrays.toString(copy));
System.out.println(java.util.Arrays.binarySearch(copy, 5));
System.out.println(java.util.Arrays.toString(data));
```

Expected result and interpretation:

```
[2, 5, 8]
1
[8, 2, 5]
Sorting the copy leaves the original order unchanged.
```

Guided problem and trace

Sort {9,1,4} without changing the original. Search the sorted result for 4 and explain the index.

1. Copy the original array before sorting.

2. Sort the copy in ascending order.

3. Search the sorted array and interpret the resulting index.

```
int[] original = {9, 1, 4};
int[] sorted = java.util.Arrays.copyOf(original, original.length);
java.util.Arrays.sort(sorted);
int index = java.util.Arrays.binarySearch(sorted, 4);
System.out.println(java.util.Arrays.toString(sorted));
System.out.println(index);
System.out.println(java.util.Arrays.toString(original));
```

Stage	Original	Copy or result
Before copy	{9,1,4}	None
After copy	{9,1,4}	{9,1,4}
After sort	{9,1,4}	{1,4,9}
Search for 4	Unchanged	Index 1

Expected output:

```
[1, 4, 9]
1
[9, 1, 4]
```

Read an API contract before composing calls

A library name is not a complete specification; mutation and failure behaviour matter too.

Relevant labels: Input assumptions; Method signature; Returned value; Edge cases

Arrays operation	Mutates input?	Important condition
sort(a)	Yes	Ascending order afterward
binarySearch(a,key)	No	Array must already be sorted
copyOf(a,n)	No	Returns a separate array

Additional worked example: Sorting library arrays

Arrays.sort changes the supplied array in place. Character sorting follows numeric UTF-16 code-unit values, not human alphabetical order. Search positions refer to the sorted arrangement, not the original arrangement.

Sort the numeric and character arrays supplied in the source example, then display them.

Input: Fixed values in the example.

```
double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};
char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};
java.util.Arrays.sort(numbers);
java.util.Arrays.sort(chars);
System.out.println(java.util.Arrays.toString(numbers));
System.out.println(java.util.Arrays.toString(chars));
```

Element kind	Order used	Example
double	Ascending numeric value	1.9 before 6.0
char	UTF-16 code-unit value	4 before A
Search after sort	Sorted index	3.4 is at index 2

Expected output:

```
[1.9, 2.9, 3.4, 3.5, 4.4, 6.0]
```

```
[4, A, D, F, P, a]
```

Independent practice

1. Sort {9,1,5}, then search for 5 using Arrays.binarySearch. Explain why searching the unsorted array is not a valid alternative.

2. How do you preserve the original order before sorting?

3. Debugging: int[] a = {8,2,5};
int index = java.util.Arrays.binarySearch(a, 2);

4. Check your understanding: Does importing Arrays sort an array? Is a negative binarySearch result a valid index?

5. Homework: Read documentation for Arrays.equals and Arrays.copyOf. Explain their signatures and demonstrate each with a short program.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_27_Library_components_and_APIs.tex and PDF/Lecture_27_Library_components_and_APIs.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture27.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Arrays1D(new).pptx, slides 35, 42, 46, 47. Adapted explanation and runnable implementation; sample inputs and traces added for this course.

Lecture 28: Files and streams

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Liang Ch 12. Learning outcome: CLO-3.

Learning objectives

- Distinguish files, paths and streams

- Inspect file metadata

- Resolve relative paths against a working directory

Concepts explained

Files and paths

A file stores persistent data outside the running program. A path identifies a location in a filesystem. Relative paths depend on the process working directory.

```
marks.txt
data/marks.txt

Both are relative paths.
```

The File class

java.io.File represents a pathname. Constructing a File object does not create the physical file. Methods such as exists and isFile inspect current metadata.

```
java.io.File f = new java.io.File("marks.txt");
boolean present = f.exists();
```

Byte and character streams

Byte streams move bytes. Character streams decode or encode text. Text encoding connects stored bytes to characters. Opening a stream acquires a resource that needs closing.

```
Text: marks and names in UTF-8
Binary: an image or audio file

Use APIs appropriate to the data.
```

File operations and failures

A missing file, a directory path and insufficient permission are different conditions. Reading and writing have different effects. Opening output can overwrite existing content.

```
Before writing:
Choose a dedicated output path.
Decide overwrite or append behaviour.
Handle the opening failure.
```

An absolute path explains where Java looks

A relative pathname resolves against the process working directory. Printing an absolute form is a useful diagnosis for a missing-file report. The same source program can resolve a relative name differently under another launcher.

```
Relative: data/marks.txt
Working directory: a course practice folder
Resolved location: that folder plus data/marks.txt
```

Metadata is an observation at one moment

exists reports whether the path exists when the call checks it. isFile distinguishes an ordinary file from a directory. A later operation still needs error handling because state or permissions can change.

```
exists=false does not create a file.
exists=true does not guarantee that a later read succeeds.
```

Key distinctions

File method	Question	Result type
getName()	Final pathname component?	String
getAbsolutePath()	Where does this path resolve?	String
exists()	Does the path currently exist?	boolean
isFile()	Does it identify a regular file?	boolean

First worked example

```
java.io.File file = new java.io.File("marks.txt");
System.out.println(file.getName());
System.out.println(file.isAbsolute());
```

Expected result and interpretation:

```
marks.txt
false
No physical file is created by the constructor.
The relative path will resolve in the program working directory.
```

Guided problem and trace

Create a File object for data/marks.txt. Display its absolute path, existence and whether it is a regular file. Explain environment-dependent output.

1. Represent data/marks.txt using a File object.

2. Print the absolute path to expose working-directory resolution.

3. Inspect existence and file type without creating or modifying the path.

```
java.io.File file = new java.io.File("data/marks.txt");
System.out.println(file.getAbsolutePath());
System.out.println(file.exists());
System.out.println(file.isFile());
```

Observation	In an empty practice folder	Reason
Absolute path	Working folder + data/marks.txt	Relative path resolution

Observation	In an empty practice folder	Reason
exists()	false	No file created
isFile()	false	Missing path is not a regular file

Expected output:

```
{WORKDIR}\data\marks.txt
false
false
```

A path is interpreted from a working directory

Constructing a File describes a path; it does not create the referenced file.

Relevant labels: Working directory; Relative path; Resolved location; File metadata

Operation	Creates a file?	What it provides
new File("marks.txt")	No	Path object
getAbsolutePath()	No	Absolute path text
exists()	No	Existence observation

Additional worked example: Creating a file is different from naming it

Constructing File creates a path object; createNewFile attempts to create storage. createNewFile returns false if the named file already exists. Creation may throw IOException; an existence check does not guarantee later creation will succeed.

Create a private temporary directory and call createNewFile twice on the same filename.

Input: Fixed values in the example.

```
java.nio.file.Path folder = java.nio.file.Files.createTempDirectory("csc103-isb-");
java.io.File file = folder.resolve("first.txt").toFile();
System.out.println(file.createNewFile());
System.out.println(file.createNewFile());
System.out.println(file.isFile());
```

Operation	Return / observation	Explanation
First createNewFile	true	New file created
Second createNewFile	false	File already exists
isFile	true	Path names a regular file

Expected output:

```
true
false
true
```

Independent practice

1. Explain why new File("marks.txt") followed by exists() may be false. Show how to print the resolved path for diagnosis.

2. Why is an absolute path not universally wrong, despite the source warning against it?

3. Debugging: new java.io.File("report.txt");
// Student expects an empty report.txt to appear.

4. Check your understanding: Does a relative path start at the source-file folder automatically? What closes a stream reliably?

5. Homework: Inspect a dedicated practice file and a directory using File methods. Record differences without deleting either.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_28_Files_and_streams.tex and PDF/Lecture_28_Files_and_streams.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture28.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Java_File_Handling__A_Beginner_s_Guide.pptx, slides 4, 5; File Handling.pptx, slides 7, 10. Uses a fresh temporary directory for safe repeatable execution; the demonstration leaves that temporary file available for inspection.

Lecture 29: Reading and writing text files

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Liang Ch 12. Learning outcome: CLO-3.

Learning objectives

- Write text with PrintWriter

- Read data with Scanner

- Use try-with-resources and explicit encoding

Concepts explained

PrintWriter output

PrintWriter writes formatted text to a destination. println writes a line and printf supports a format string. Opening the demonstrated writer replaces existing file contents.

```
try (java.io.PrintWriter out =
    new java.io.PrintWriter("demo.txt", "UTF-8")) {
  out.println(60);
  out.println(80);
}
```

Scanner file input

Scanner can parse a text file into tokens. hasNextInt checks whether the next token matches an integer. Reading must advance even when a token is invalid.

```
try (java.util.Scanner in =
    new java.util.Scanner(new java.io.File("demo.txt"), "UTF-8")) {
  while (in.hasNextInt()) {
    System.out.println(in.nextInt());
  }
}
```

Try-with-resources

Declare resources in the try header. Java closes them when the block exits. Multiple resources close in reverse declaration order.

```
try (Resource r = openResource()) {
  // use r
}
// r closes on normal or exceptional exit
```

A file-processing contract

Define the encoding and record format. Choose what malformed input means: reject it or report and skip it. Treat empty files and missing files as separate cases.

```
Contract for this example:
UTF-8 integer tokens
Malformed token: reject the file
```

```
Empty file: sum is zero
```

A reader must consume or reject every token

A loop controlled by hasNext continues while a token exists. If the token is not an integer, the program must consume it or end with a failure. Doing neither leaves the same token waiting forever.

```
while (in.hasNext()) {
  if (!in.hasNextInt()) throw ...;
  int mark = in.nextInt();
}
```

The empty case differs from malformed data

An empty file has no marks and therefore no mean. A malformed token violates the declared integer format. Handling these as different outcomes gives a useful explanation to the caller.

```
Empty file: No data
"60 80": average 70.0
"60 bad": reject malformed token
```

Key distinctions

File contents	Interpretation	Required outcome
60 80	Two valid marks	Average 70.0
Empty	Zero records	No data
60 bad	Malformed token	Reject with token context
60 101	Out-of-range mark	Reject range violation

First worked example

```
String path = "lecture29-demo.txt";
try (java.io.PrintWriter out = new java.io.PrintWriter(path, "UTF-8")) {
  out.println(60); out.println(80);
}
int sum = 0;
try (java.util.Scanner in = new java.util.Scanner(new java.io.File(path), "UTF-8")) {
  while (in.hasNextInt()) sum += in.nextInt();
}
System.out.println(sum);
```

Expected result and interpretation:

```
140
The writer closes before the reader opens.
The dedicated demonstration file contains 60 and 80.
```

Guided problem and trace

Read a file of integer marks and reject the first malformed token. Print No data for an empty file, otherwise the average.

1. Create a dedicated UTF-8 fixture containing 60 and 80 for the demonstration.

2. Read tokens inside try-with-resources, rejecting malformed or out-of-range values.

3. Compute a mean only when count is positive, and check for a recorded Scanner I/O failure.

```
String path = "expanded29-demo.txt";
try (java.io.PrintWriter out = new java.io.PrintWriter(path, "UTF-8")) {
  out.println("60 80");
}
reportAverage(path);
```

```
static void reportAverage(String path) throws Exception {
  long sum = 0;
  int count = 0;
  try (java.util.Scanner in = new java.util.Scanner(
      new java.io.File(path), "UTF-8")) {
    while (in.hasNext()) {
      if (!in.hasNextInt()) {
        throw new IllegalArgumentException("Bad token: " + in.next());
      }
      int mark = in.nextInt();
      if (mark < 0 || mark > 100) {
        throw new IllegalArgumentException("Mark outside 0..100");
      }
      sum += mark;
      count++;
    }
    if (in.ioException() != null) throw in.ioException();
  }
  if (count == 0) System.out.println("No data");
  else System.out.println((double) sum / count);
}
```

Next token	Action	Sum	Count
60	Parse and validate	60	1
80	Parse and validate	140	2
End of file	Stop reading	140	2
Report	140 / 2.0	70.0 mean	2

Expected output:

```
70.0
```

Resources have a controlled lifetime

Try-with-resources attempts closure when the body exits normally or by exception.

Relevant labels: Open resources; Read or write; Close resources; Continue / propagate

File content	Required action	Reason
60 80	Report mean 70.0	Two valid marks
60 bad	Reject input	Do not silently skip malformed data
Empty	Report No data	No defined mean

Additional worked example: Reading structured student records

The source format has first name, middle initial, surname and score on each record. Treat the fourth field consistently as a score; the source reader labels it age despite score-writing examples. Read and validate each line independently so malformed records cannot silently shift the remaining fields.

Write John T Smith 90 and Eric K Jones 85, then read and print each name and score.

Input: Fixed values in the example.

```
java.nio.file.Path file = java.nio.file.Files.createTempFile("isb-records-", ".txt");
try (java.io.PrintWriter out = new java.io.PrintWriter(file.toFile(), "UTF-8")) {
  out.println("John T Smith 90");
  out.println("Eric K Jones 85");
  if (out.checkError()) throw new java.io.IOException("write failed");
}
try (java.util.Scanner in = new java.util.Scanner(file, "UTF-8")) {
  while (in.hasNextLine()) {
    String[] fields = in.nextLine().trim().split("\\s+");
    if (fields.length != 4) throw new IllegalArgumentException("four fields required");
    int score = Integer.parseInt(fields[3]);
    System.out.println(fields[0] + " " + fields[2] + ": " + score);
  }
  if (in.ioException() != null) throw in.ioException();
}
```

Record	Parsed name	Score
John T Smith 90	John Smith	90
Eric K Jones 85	Eric Jones	85
Missing fourth field	Rejected	No score

Expected output:

```
John Smith: 90
Eric Jones: 85
```

Independent practice

1. Write a short try-with-resources fragment that scans all integer tokens from marks.txt. Reject a non-integer token instead of ending silently.

2. Which further check is required if scores must be in 0..100?

3. Debugging: while (in.hasNext()) {
if (in.hasNextInt()) sum += in.nextInt();
}

4. Check your understanding: Why close the writer before reading its output? What happens to existing data when this writer opens?

5. Homework: Write three marks to a practice file and read them back. Test an empty file, malformed text and a missing input path.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_29_Reading_and_writing_text_files.tex and PDF/Lecture_29_Reading_and_writing_text_files.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture29.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: File Handling.pptx, slides 14, 15, 18. Used UTF-8, try-with-resources and line-based field-count checks; retained the source names and scores.

Lecture 30: Test harnesses

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Reference material. Learning outcome: CLO-4.

Learning objectives

- Build a repeatable test harness

- Choose expected results independently

- Report passes and failures meaningfully

Concepts explained

Tests and oracles

A test supplies input and checks observable behaviour. An oracle specifies the expected outcome. Calculate expected results independently of the implementation.

```
Function: max(a,b)
Input: (-4,-2)
Expected: -2
Observed: result returned by the method
```

A simple harness

A harness calls the target repeatedly and reports results. Each failure should identify its case and expected value. A summary helps detect missed or skipped cases.

```
Case: negative pair
Expected: -2
Actual: 0
Status: FAIL
```

Equivalence and boundaries

Group inputs that should behave similarly. Test transitions between groups and exceptional inputs. Include empty or zero-sized input where the contract permits it.

```
For isPass(mark):
49, 50, 51
For a valid-mark contract:
-1, 0, 100, 101
```

Repeatability and regression

Tests should produce the same result under the same conditions. Control random seeds, data files and other changing dependencies. Keep a small test that exposes each fixed bug.

```
Regression case:
Input: two negative numbers
Expected: larger negative number
Purpose: detect zero initialisation bug
```

The expected result must be independent

A harness compares actual behaviour to an oracle derived from the specification. Using the same potentially faulty implementation on both sides proves little. Literal expected results work well for a small set of manually solved cases.

```
Input -5 has expected absolute value 5.
The expected value comes from the mathematical definition.
```

Numeric limits belong in the contract

Integer.MIN_VALUE has no positive counterpart representable as int. Returning long lets an absolute-value function represent that magnitude. The conversion must happen before negation to avoid int overflow.

```
long widened = value;
return widened < 0 ? -widened : widened;
```

Key distinctions

Input	Expected long result	Reason for case
-5	5	Negative ordinary input
0	0	Zero boundary
5	5	Positive ordinary input
Integer.MIN_VALUE	2147483648	Minimum int magnitude

First worked example

```
int[][] cases = {{2, 5, 5}, {-4, -2, -2}, {3, 3, 3}};
int passed = 0;
for (int[] c : cases) {
  int actual = max(c[0], c[1]);
  if (actual == c[2]) passed++;
  else System.out.println("FAIL expected " + c[2] + " got " + actual);
}
System.out.println("Passed " + passed + "/" + cases.length);

static int max(int a, int b) { return a >= b ? a : b; }
```

Expected result and interpretation:

```
Passed 3/3
Each row contains first input, second input and expected result.
The equal-input case checks ties.
```

Guided problem and trace

Create a harness for absolute value over -5, 0 and 5. State what your method does for Integer.MIN_VALUE.

1. Store manually calculated expected results in long values.

2. Call the absolute-value method for each input.

3. Print expected and actual values with a clear pass or fail label.

```
int[] inputs = {-5, 0, 5, Integer.MIN_VALUE};
long[] expected = {5L, 0L, 5L, 2147483648L};
for (int i = 0; i < inputs.length; i++) {
  long actual = absolute(inputs[i]);
  String status = actual == expected[i] ? "PASS" : "FAIL";
  System.out.println(status + " " + inputs[i] + " => " + actual);
}
```

```
static long absolute(int value) {
  long widened = value;
  return widened < 0 ? -widened : widened;
}
```

Input	Widened value	Negation needed?	Actual result
-5	-5L	Yes	5L
0	0L	No	0L
5	5L	No	5L
Minimum int	-2147483648L	Yes	2147483648L

Expected output:

```
PASS -5 => 5
PASS 0 => 0
PASS 5 => 5
PASS -2147483648 => 2147483648
```

A test harness compares two independent paths

The expected result must come from the specification, not from calling the same implementation again.

Relevant labels: Chosen input; Program result; Expected result; Compare and report

Test	Expected max	Fault it can expose
max(3,7)	7	Wrong comparison
max(-3,-7)	-3	Incorrect zero initialisation
max(5,5)	5	Mishandled equality

Additional worked example: A harness for the digit-sum exercise

Turn the source example into a method contract with ordinary and boundary cases. An expected value must be derived independently of the method under test. A harness should fail visibly when a comparison fails.

Check digit sums for 234, 0 and 1001 with explicit comparisons; report how many pass.

Input: Fixed values in the example.

```
long[] inputs = {234, 0, 1001};
int[] expected = {9, 0, 2};
for (int i = 0; i < inputs.length; i++) {
  int actual = sumDigits(inputs[i]);
  if (actual != expected[i]) throw new AssertionError("case " + i);
```

```
}
System.out.println("Passed 3/3");

static int sumDigits(long n) {
  if (n < 0) throw new IllegalArgumentException("nonnegative required");
  int sum = 0;
  while (n != 0) { sum += n % 10; n /= 10; }
  return sum;
}
```

Input	Expected sum	Reason
234	9	Ordinary source case
0	0	Zero boundary
1001	2	Internal zero digits

Expected output:

```
Passed 3/3
```

Independent practice

1. Build a small explicit check for max(-3,-7). Explain why Java assert statements can be an unreliable default harness without the required runtime option.

2. Would a method that always returns 9 pass this harness?

3. Debugging: int expected = buggyMax(a,b);
int actual = buggyMax(a,b);

4. Check your understanding: What makes a failed test useful? Why include a tie case for maximum?

5. Homework: Build a harness with at least six cases for your average or maximum method and retain one deliberate bug demonstration.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_30_Test_harnesses.tex and PDF/Lecture_30_Test_harnesses.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture30.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Methods Practice.pptx, slides 2, 3. Instructor-authored testing adaptation of the supplied digit-sum exercise; the source does not contain this harness.

Source exercise deck credits Instructor Khurram Iqbal.

Lecture 31: Unit testing with JUnit

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Reference material: JUnit Jupiter. Learning outcome: CLO-4.

Learning objectives

- Write a focused unit test

- Use equality and exception assertions

- Separate test setup, action and checks

Concepts explained

A unit test

A unit test checks a small unit of behaviour, usually through a method. A test name explains the case and expected outcome. Keep the test independent of other tests.

```
Test name: largerOfTwoNegatives
Given: -4 and -2
Expected: -2
```

JUnit structure

@Test marks a Jupiter test method. Assertions fail the test when a result violates the expectation. Production code and test code belong in separate source folders.

```
@Test
void largerOfTwoNegatives() {
  assertEquals(-2, Numbers.max(-4, -2));
}
```

Assertions for behaviour

assertEquals compares an expected result to the actual result. assertTrue checks a condition. assertThrows checks an exception contract. For approximate doubles, choose a justified tolerance.

```
assertEquals(2.5, mean(2,3), 1e-9);
assertThrows(IllegalArgumentException.class,
             () -> validateMark(-1));
```

Setup, action and checks

Arrange the input and required state. Act by calling the target. Assert the outcome without depending on a previous test.

```
Arrange: int[] a = {2,3};
Act: int[] b = doubled(a);
Assert: b is {4,6}; a is still {2,3}.
```

Exception assertions execute an action

assertThrows receives an expected exception type and an action to execute. The action must be deferred so the assertion can observe its failure. A lambda supplies that action without running it during argument preparation.

```
assertThrows(IllegalArgumentException.class,
    () -> validateMark(101));
```

A passing boundary test checks inclusion

The upper allowed value, 100, should return normally. The next value, 101, should throw the documented exception. Testing both distinguishes inclusive and exclusive range implementations.

```
Valid condition: 0 <= mark && mark <= 100
Reject condition: mark < 0 || mark > 100
```

Key distinctions

Assertion	Checks	Typical use
assertEquals	Expected and actual values	A numerical result
assertArrayEquals	Element contents	An array transformation
assertThrows	Expected exception type	Invalid input
assertDoesNotThrow	Normal completion	A valid boundary input

First worked example

```
int result = max(-4, -2);
if (result != -2) throw new AssertionError("negative pair");
System.out.println("Unit behaviour passed");

static int max(int a, int b) { return a >= b ? a : b; }
```

Expected result and interpretation:

```
Unit behaviour passed
The companion JUnit project expresses this same check with @Test and assertEquals.
This console demonstration has no external dependency.
```

Guided problem and trace

Write a Jupiter test asserting that a mark validator rejects 101 and accepts 100.

1. Use the same validator contract as the production method.

2. Assert that 101 throws IllegalArgumentException.

3. Assert that 100 completes without throwing.

```
org.junit.jupiter.api.Assertions.assertThrows(
    IllegalArgumentException.class, () -> validateMark(101));
org.junit.jupiter.api.Assertions.assertDoesNotThrow(
    () -> validateMark(100));
System.out.println("2 checks passed");

static void validateMark(int mark) {
  if (mark < 0 || mark > 100) {
    throw new IllegalArgumentException("mark outside 0..100");
  }
}
```

Input	Expected behaviour	Assertion	Result
101	Throws IllegalArgumentException	assertThrows	Pass
100	Returns normally	assertDoesNotThrow	Pass

Expected output:

```
2 checks passed
```

A unit test separates setup, action and check

A passing assertion is evidence about its chosen case, not proof for every input.

Relevant labels: Arrange inputs; Act: call method; Assert result; Runner reports

Test input	Expected behaviour	JUnit assertion
mark 100	Accepted	assertDoesNotThrow
mark 101	Rejected	assertThrows
mean of 40 and 60	50.0	assertEquals with tolerance

Additional worked example: JUnit assertions for digit processing

Reuse the same digit-sum contract in automated tests. Assert the returned value for valid cases and the exception type for rejected cases. JUnit assertions run when called; @Test methods are discovered by the test runner in the companion project.

Check sumDigits(234), sumDigits(0), and rejection of a negative input using Jupiter assertions.

Input: Fixed values in the example.

```
org.junit.jupiter.api.Assertions.assertEquals(9, sumDigits(234));
org.junit.jupiter.api.Assertions.assertEquals(0, sumDigits(0));
org.junit.jupiter.api.Assertions.assertThrows(
  IllegalArgumentException.class, () -> sumDigits(-1));
System.out.println("JUnit checks passed");
```

```
static int sumDigits(long n) {
  if (n < 0) throw new IllegalArgumentException("nonnegative required");
  int sum = 0;
  while (n != 0) { sum += n % 10; n /= 10; }
  return sum;
}
```

Input	Expected behaviour	Assertion
234	Returns 9	assertEquals
0	Returns 0	assertEquals
-1	Rejected	assertThrows

Expected output:

```
JUnit checks passed
```

Independent practice

1. Write a Jupiter test for mean({40,60}) using the supplied Numbers class. Identify the expected value and tolerance.

2. What assertion would check the internal-zero case without changing production code?

3. Debugging: @Test void test() {
Numbers.max(2,5);
}

4. Check your understanding: Does an assertion-free test prove a returned value is correct? Why use assertArrayEquals?

5. Homework: Add JUnit tests for negative values, ties and invalid input to the supplied Numbers example.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_31_Unit_testing_with_JUnit.tex and PDF/Lecture_31_Unit_testing_with_JUnit.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture31.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: Methods Practice.pptx, slides 2, 3. Instructor-authored JUnit adaptation, not a claim that the supplied slides teach JUnit. Runnable with Jupiter on the classpath.

Source exercise deck credits Instructor Khurram Iqbal.

Lecture 32: Testing practice and course synthesis

Student handout | CSC103 Programming Fundamentals | Fall 2026

Reading: Reference material: JUnit Jupiter. Learning outcome: CLO-4.

Learning objectives

- Organise parameterised and regression tests

- Distinguish unit and integration checks

- Test a program that reads and summarises marks

Concepts explained

Parameterised tests

A parameterised test applies one behaviour check to several cases. @CsvSource supplies small literal datasets. Each case should add a reason to test.

```
@ParameterizedTest
@CsvSource({"2,5,5", "-4,-2,-2", "3,3,3"})
void maximumCases(int a, int b, int expected) {
  assertEquals(expected, Numbers.max(a,b));
}
```

Isolation and fixtures

Tests should avoid shared mutable state. Temporary directories isolate file tests. Resource cleanup matters even when an assertion fails.

```
@TempDir Path temp;
Path file = temp.resolve("marks.txt");
// Write a controlled fixture for this test.
```

Testing an integrated program

Separate parsing, validation and calculation. Test pure calculations with small unit tests. Use file fixtures to check the whole reading-and-summary path.

```
Fixture: "40 50 60 70"
Expected mean: 55.0
Expected passes: 3
Invalid fixture: "50 bad 60"
```

Coverage and final review

Line coverage says which lines executed, not whether outcomes were checked well. A small changed condition can reveal weak tests. Review algorithms, control flow, arrays and error handling together.

```
Mutation thought experiment:
Change mark >= 50 to mark > 50.
Which test must fail?
Answer: the case at exactly 50.
```

A test matrix connects requirements to evidence

Each requirement should have an observable result and a corresponding test. Boundary cases expose comparisons that ordinary values may never challenge. Failure cases should check the intended exception or message.

```
Requirement: 50 counts as a pass.
Test input: {50}
Expected passing count: 1
```

A file fixture isolates an integration test

A fixture supplies controlled input data for a test. A temporary directory prevents collisions with a user's real files. The test then checks the result of file reading and calculation together.

```
Write "40 50 60 70" to a temporary file.
Read it through the production method.
Assert mean 55.0 and passing count 3.
```

Key distinctions

Requirement	Test input	Expected evidence
Mean calculation	40 50 60 70	55.0
One-record handling	80	Mean 80.0
Passing threshold	50	Pass count 1
Empty mean policy	Empty file	Exception for mean
Format validation	50 bad 60	Parsing-related exception
Range validation	101	Range exception

First worked example

```
int[] marks = {40, 50, 60, 70};
int sum = 0, passes = 0;
for (int mark : marks) {
  sum += mark;
  if (mark >= 50) passes++;
}
System.out.println((double) sum / marks.length);
System.out.println(passes);
```

Expected result and interpretation:

```
55.0
3
A boundary test at 50 distinguishes >= from >.
```

Guided problem and trace

Design tests for a marks-file summary: normal data, one mark, threshold 50, empty file, malformed token and out-of-range mark.

1. Use a pure mean method that rejects empty or out-of-range data.

2. Use a pass-count method with the inclusive threshold 50.

3. Compare a normal dataset with a single mark exactly on the boundary.

```
int[] marks = {40, 50, 60, 70};
System.out.println(mean(marks));
System.out.println(passes(marks));
System.out.println(passes(new int[]{50}));

static double mean(int[] marks) {
  if (marks.length == 0) throw new IllegalArgumentException("empty");
  long sum = 0;
  for (int mark : marks) {
    if (mark < 0 || mark > 100) throw new IllegalArgumentException("range");
    sum += mark;
  }
  return (double) sum / marks.length;
}
static int passes(int[] marks) {
  int count = 0;
  for (int mark : marks) if (mark >= 50) count++;
  return count;
}
```

Test	Expected	If >= becomes >	Detects change?
Passes in {40,50,60,70}	3	2	Yes
Passes in {80}	1	1	No
Passes in {50}	1	0	Yes

Expected output:

```
55.0
3
1
```

Integration tests cross component boundaries

An integration test checks that collaborating components agree on data and behaviour.

Relevant labels: Temporary file; Parser; Summary methods; Assertions

Fixture	Expected outcome	Boundary exercised
40 60	Mean 50.0; passes 1	File through summary
50 bad	Parsing failure	Malformed file token
Empty file	Empty array; mean rejected	No-data contract

Additional worked example: Integration checks for student-record files

A temporary fixture isolates the test from personal files and previous runs. Check parsing separately from an aggregate calculation. Malformed fields and out-of-range scores are distinct failure cases.

Write the two source records to a temporary UTF-8 file, parse the scores, and assert scores {90,85} and mean 87.5.

Input: Fixed values in the example.

```
java.nio.file.Path file = java.nio.file.Files.createTempFile("isb-test-", ".txt");
java.nio.file.Files.writeString(file, "John T Smith 90\nEric K Jones 85\n",
  java.nio.charset.StandardCharsets.UTF_8);
int[] scores = readScores(file);
org.junit.jupiter.api.Assertions.assertArrayEquals(new int[]{90, 85}, scores);
double mean = (scores[0] + scores[1]) / 2.0;
org.junit.jupiter.api.Assertions.assertEquals(87.5, mean, 1e-9);
System.out.println("Record integration passed");
```

```
static int[] readScores(java.nio.file.Path file) throws java.io.IOException {
  java.util.List<String> lines = java.nio.file.Files.readAllLines(
    file, java.nio.charset.StandardCharsets.UTF_8);
  int[] scores = new int[lines.size()];
  for (int i = 0; i < lines.size(); i++) {
    String[] fields = lines.get(i).trim().split("\\s+");
    if (fields.length != 4) throw new IllegalArgumentException("four fields required");
    int score = Integer.parseInt(fields[3]);
    if (score < 0 || score > 100) throw new IllegalArgumentException("score range");
    scores[i] = score;
  }
  return scores;
}
```

Fixture	Expected parsing	Expected next step
Two source records	[90, 85]	Mean 87.5
John T Smith bad	Reject non-integer	No aggregate
John T Smith 101	Reject range	No aggregate

Expected output:

```
Record integration passed
```

Independent practice

1. Plan an integration test for a temporary file containing 40 and 60. List the assertions that distinguish successful reading from successful calculation.

2. Why is checking only mean 87.5 weaker than also checking the parsed array?

3. Debugging: A suite executes every line, but never checks the calculated average. Is the calculation verified?

4. Check your understanding: Which tests should fail if >=50 becomes >50? What does a file integration test cover beyond a pure unit test?

5. Homework: Prepare a small marks-analysis project with methods, arrays, file I/O and a documented test suite. Use the companion project as a starting point.

For each programming task, state the input assumptions, predict a result, then compare it with execution. Record a normal case and a boundary case; explain invalid-input behavior where it is part of the task.

Sources and runnable examples

Companion slides: ISB_Enhanced_Beamer/Lecture_32_Testing_practice_and_course_synthesis.tex and PDF/Lecture_32_Testing_practice_and_course_synthesis.pdf. Java snippets above may be method bodies; use the complete companion files to run them.

Complete additional example: ISB_Enhanced_Beamer/ISB_Java_Examples/IsbLecture32.java. Lectures 31-32 also use the companion JUnit_Project. The source course targets Java 17 or later.

Supplied ISB sources: File Handling.pptx, slides 14, 15, 18. Instructor-authored integration-test extension of the source records. List<String> is a supporting library use, not a new assessed data-structure unit.